GCP Interview Questions and Answers

1. **What is big query? Can Big query store Unstructured data?**
   Big Query is a fully-managed, serverless, and highly scalable data warehouse offered by Google Cloud Platform (GCP). It is designed for fast SQL-based querying and analysis of large datasets (petabyte-scale) using the power of Google's infrastructure. It supports real-time analytics, machine learning integrations, and BI tools like Looker and Data Studio.
   Key Features of BigQuery:

- Serverless and fully managed
- Uses standard SQL
- Supports partitioning and clustering for performance
- Integrates with GCP services like Cloud Storage, Cloud Composer, Dataflow
- Offers built-in machine learning with BigQuery ML
- Supports streaming and batch data ingestion

Can Big Query Store Unstructured Data?
No, Big Query is primarily designed for structured and semi-structured data, not unstructured data.

2. **Google Big Query Archtechture ?**

Google Bigquery uses a **serverless, distributed architecture** built on top of **Dremel technology**, optimized for fast querying and high scalability. Here's a simplified breakdown of its architecture:

🔧 **Main Components:**

1. Storage 🗄️

- Stores your data in columnar format (more efficient than row-wise).
- Data is stored in Google Cloud Storage (behind the scenes).
- Fully managed: You don't worry about how it's stored or indexed.

2. Query Engine (Dremel) ⚙️

- This is the brain of BigQuery.
- **Massively parallel processing:** Splits the query and runs it across many servers at once.
- Based on Dremel technology, which makes queries super-fast even on petabytes of data.

3. Slots (Compute Units) 🧱

- These are the resources used to run your query.
- You can think of slots like "workers" that help process parts of the query.
- BigQuery assigns slots automatically (or you can reserve them).

4.Jobs 📦

- A job is any task you ask BigQuery to do — like running a query, loading data, or exporting.
- Jobs are queued and managed automatically.

5. Metadata & Catalog 📚

- Keeps track of datasets, tables, schemas, and permissions.
- Managed by BigQuery internally.

🎯 **Key Benefits:**

- No infrastructure to manage
- Scales automatically
- Pay only for what you use
- Fast performance on huge datasets

3. **What is the Architecture of Big query?**
   Google BigQuery uses a **serverless, distributed architecture** built on top of **Dremel technology**, optimized for fast querying and high scalability. Here's a simplified breakdown of its architecture:

🧱 **1. Storage Layer (Colossus)**

- BigQuery separates **storage** from **compute**.
- Data is stored in **Colossus**, Google's distributed file system.
- It stores data in a **columnar format** for high performance and compression.
- Supports **partitioning** and **clustering** for efficient access.

🧠 **2. Compute Layer (Dremel Execution Engine)**

- The compute engine is based on **Dremel**, a highly scalable query engine.
- It **distributes and parallelizes** queries across thousands of machines.
- Each SQL query is broken down into **tree-based stages** for optimized execution.
- Auto-scales based on query size—**no infrastructure management** needed.

🔗 **3. Query Interface & SQL Engine**

- Users interact with BigQuery using **Standard SQL**, APIs, or client libraries.
- Supports **ad hoc querying**, **batch jobs**, **streaming**, and **materialized views**.
- **BigQuery ML** allows SQL-based machine learning directly in the platform.

🛠️ **4. Ingestion & Integration**

- Supports data ingestion from:
  - **Batch loads** (CSV, JSON, Parquet, Avro from GCS)
  - **Streaming inserts** (real-time data pipelines)
  - **Federated queries** (e.g., querying external sources like Google Sheets, Cloud Storage, Bigtable)
  - **Data Transfer Service** for scheduling regular data imports

---

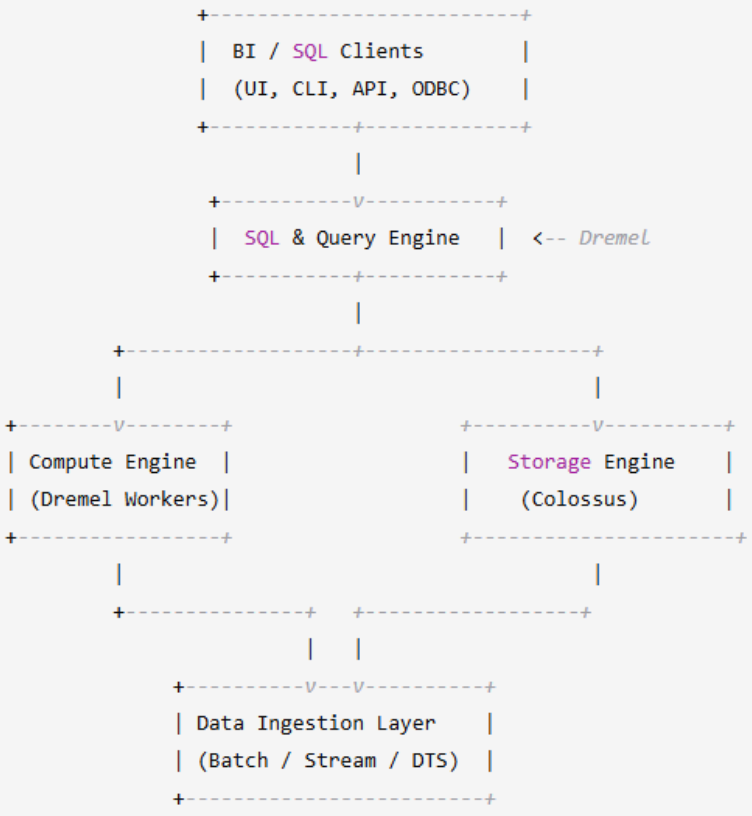
📊 **5. Tools & Ecosystem**

- Works with GCP ecosystem:
  - **Dataflow, Cloud Composer (Airflow), Pub/Sub, Dataform, Dataproc**
- Integrates with BI tools:
  - **Looker, Data Studio, Power BI, Tableau**
- Supports **access controls** via IAM and **auditing** via Cloud Logging

📌 BigQuery Architecture Diagram (Text Representation)

pgsql

Copy Edit



4. **how is bigquery different from another traditional database?**
   BigQuery is **fundamentally different** from traditional databases like Oracle, MySQL, or PostgreSQL in several important ways. Here's a comparison highlighting the **key differences**:

🔍 1. Architecture & Infrastructure

Aspect	BigQuery	Traditional Databases
Type	Serverless, fully managed	Self-managed or cloud-hosted
Infrastructure	Runs on Google Cloud infrastructure (distributed)	Runs on a single machine or cluster (user managed)
Storage & Compute	Decoupled (scale independently)	Typically tightly coupled

⚡ 2. Scalability & Performance

Aspect	BigQuery	Traditional Databases
Scalability	Massively parallel processing (MPP)	Limited by hardware or cluster size
Performance	Optimized for **large-scale analytics** (terabytes to petabytes)	Slower for very large datasets unless tuned manually
Auto-scaling	Yes, automatically scales with data	Manual tuning needed

⏱️ 3. Query Execution & Cost

Aspect	BigQuery	Traditional Databases
Query Engine	Columnar, distributed SQL engine	Row-based or partially columnar
Pricing Model	Pay-per-query (based on data scanned)	Pay for server/resources regardless of usage
Cost Control	On-demand or flat-rate	Fixed cost, hardware/license cost

💾 4. Data Handling

Aspect	BigQuery	Traditional Databases
Schema	Supports semi-structured data (e.g., JSON) natively	Limited support; often requires normalization
Data Load	Easily integrates with GCS, streaming, or batch loads	ETL usually required
Data Volume	Optimized for analytical workloads over huge volumes	Designed for transactional workloads over smaller volumes

🔐 5. Maintenance & Management

Aspect	BigQuery	Traditional Databases
Maintenance	No indexing, vacuuming, patching needed	Manual tuning, indexing, backups required
Security	Built-in IAM, encryption, VPC Service Controls	Must be configured manually
Backup/Restore	Managed by GCP with time travel & snapshots	Manual or semi-automated setup required

🔄 6. Use Cases

Aspect	BigQuery	Traditional Databases
Best for	Analytics, dashboards, machine learning, large-scale data warehouse	OLTP (Online Transaction Processing), CRUD apps
Not ideal for	Frequent row-level updates or deletes	Massive data aggregation/analytics

5. **What are the Optimation techniques in Big query? (Partitioning and Clustering).**

✅ **What is Partitioning?**

Partitioning means **dividing a big table into smaller parts** based on a column (usually a **date** column).

This helps BigQuery scan **only the required data** — which makes the query **faster** and **cheaper**.

🔹 **Example:**

If your table has sales data from 2020 to 2025, and you only query data from 2024, BigQuery will scan **only 2024 data** — not the whole table.

📌 **Types of Partitioning:**

- By **Date or Timestamp**
- By **Ingestion Time** (when the data was loaded)
- By **Integer Range**

✅ **What is Clustering?**

Clustering means **organizing data inside each partition** (or whole table) based on specific columns like customer_id, region, or status.

It helps BigQuery find data **faster** when you filter using those columns.

🔹 **Example:**

If your table is clustered by customer_id, and your query is only for customer_id = 123, BigQuery will **directly jump to that data** — no need to scan all rows.

✅ **When to Use:**

Feature	Use it when you...
Partitioning	Often query by **date/time**
Clustering	Often filter by **columns like ID or region**

✅ **Benefits:**

- Faster queries
- Lower cost (less data scanned)
- Better performance

6. **how do you trigger an Airflow DAG from another DAG?**

To trigger one Airflow DAG from another DAG, you can use the **TriggerDagRunOperator**. This operator allows you to programmatically trigger another DAG from within your current DAG.

✅ **Example: Trigger a DAG from Another DAG**

Let's say you have:

- **Parent DAG**: parent_dag
- **Child DAG**: child_dag (the one you want to trigger)

Code for parent_dag.py:

```
python
CopyEdit
from airflow import DAG
from airflow.operators.dagrun_operator import TriggerDagRunOperator
from datetime import datetime

with DAG (
   dag_id="parent_dag",
   start_date=datetime (2024, 1, 1),
   schedule_interval=None,
   catchup=False,
) as dag:

   trigger_child = TriggerDagRunOperator(
      task_id="trigger_child_dag",
      trigger_dag_id="child_dag",  # The DAG ID of the child DAG
      wait_for_completion=False,   # Set to True if you want the parent DAG to wait for the
child to finish
   )
```

🔁 **Optional: Pass Parameters to Child DAG**

You can pass data to the child DAG using conf:

```
python
CopyEdit
trigger_child = TriggerDagRunOperator (
   task_id="trigger_child_dag",
   trigger_dag_id="child_dag",
   conf={"key": "value"},  # parameters passed to the child DAG
   wait_for_completion=False,
)
```

In the child DAG, you can access these values using:

python
CopyEdit
dag_run.conf.get("key")

📌 **Notes:**

- The child_dag must be **independent** (not a SubDAG).
- Both DAGs must be present in your dags/ folder so that Airflow knows about them.
- You can use **wait_for_completion=True** if you want the parent DAG to pause until the child finishes.

7. **What are the ways of loading data to Bigquery Table?**

✅ Ways to Load Data into BigQuery Table

Method	Description
1. Manual Upload	Upload files (CSV, JSON, Avro, etc.) directly through the BigQuery UI.
2. Batch Load	Load large files from Google Cloud Storage (GCS) using SQL or UI.
3. Streaming Insert	Real-time loading of one row or multiple rows using API or tools like Pub/Sub + Dataflow.
4. Scheduled Data Transfer	Use BigQuery Data Transfer Service (DTS) to automatically pull data from sources like Google Ads, Google Analytics, YouTube, SaaS apps.
5. Federated Query (External Table)	Query data directly from GCS, Google Drive, or Bigtable without loading into BigQuery.
6. ETL/ELT Tools	Use tools like Cloud Dataflow, Apache Airflow (Composer), Dataform, Informatica, or DBT to load, transform, and manage data pipelines.
7. BigQuery API / SDK	Use Python, Java, or REST API to automate data loads programmatically.
8. Google Sheets Connector	Connect and load data directly from Google Sheets into BigQuery using the UI or BigQuery Connector for Sheets.

✅ **Summary Table**

Load Method	Best For	Speed	Cost Efficient
Manual Upload	Small, one-time loads	Medium	✅ Yes

Load Method	Best For	Speed	Cost Efficient
Batch Load from GCS	Large historical data loads	Fast	✅ Yes
Streaming Inserts	Real-time data	Very Fast	❌ (More Cost)
Data Transfer Service	Scheduled loads from other tools	Scheduled	✅ Yes
Federated Query	Reading data without loading	On Demand	✅ Yes
ETL Tools	Complex transformation & pipelines	Scheduled/Real-time	✅ Yes
API/SDK	Automated custom loads	Customizable	✅ Yes
Google Sheets	Lightweight reporting use cases	Medium	✅ Yes

8. **How to retrieve deleted table in Big query ?**
   🕰️ **What is Time Travel in BigQuery?**
   BigQuery **automatically retains deleted or changed table data** for **7 days**.
   You can access the **snapshot** of a table **as it existed at a specific point in the past** using **FOR SYSTEM_TIME AS OF**.

   ✅ **Steps to Recover a Deleted Table:**
   🔹 1. Find the deleted table name and dataset
   You need to know:

- Project ID
- Dataset name
- Deleted table name
- Timestamp before the deletion (format: YYYY-MM-DD HH:MM:SS)

Use SQL to restore the deleted table from Time Travel:

```
CREATE OR REPLACE TABLE project_id. dataset.restored_table AS
SELECT * FROM `project_id.dataset.deleted_table`
FOR SYSTEM_TIME AS OF TIMESTAMP("2025-06-09 10:30:00");
```

9. **How we can avoid Table to get deleted in Bigquery?**
   To avoid a table from getting deleted in BigQuery, you can follow these best practices and technical safeguards:

✅ **1. Use Table-Level IAM Permissions**

Limit who can delete tables by using Identity and Access Management (IAM).

- Do not assign roles/bigquery.dataOwner or roles/bigquery.admin to users unless absolutely necessary.
- Assign roles/bigquery.dataViewer or roles/bigquery.dataEditor instead to restrict delete access.

Command to set role:

bash

CopyEdit

```
gcloud projects add-iam-policy-binding [PROJECT_ID] \
  --member=user:[USER_EMAIL] \
  --role=roles/bigquery.dataViewer
```

✅ **2. Enable Table Expiration Control Carefully**

Avoid using table expiration unless necessary.

- If expiration_time is set, the table is automatically deleted after that time.
- You can remove expiration using SQL:

sql

CopyEdit

```
ALTER TABLE `project. dataset. Table`
SET OPTIONS (expiration_timestamp = NULL);
```

✅ **3. Use Labels and Monitoring for Audit**

Add labels like {"env":"prod", "protected":"true"} and monitor logs using Cloud Audit Logs.

- Track DeleteTable events in logs.
- Set up alerts via Cloud Monitoring if deletion is attempted.

✅ **4. Backup Critical Tables**

Schedule regular exports to Cloud Storage (CSV, JSON, Avro, Parquet).

bash

CopyEdit

```
bq extract --destination_format=CSV project:dataset.table gs://your-bucket/backup.csv
```

You can automate this using Cloud Scheduler + Cloud Functions or Composer (Airflow).

✅ **5. Use Views or Authorized Views Instead of Sharing Full Table Access**

If others only need to query data, provide access to views instead of the base table.

✅ **6. Restrict Dataset Permissions**

Manage access at the dataset level by:

- Removing "Editor" and "Owner" roles from users.
- Giving read-only access where possible.

✅ **Summary Table**

Method	Description
IAM Roles Restriction	Avoid giving delete access

Method	Description
Remove Expiration Time	Prevent auto-deletion of tables
Enable Audit Logs	Track deletion attempts
Regular Backups	Restore if deletion occurs
Use Views	Avoid giving full table access
Dataset-Level Permissions	Reduce risk at dataset scope

10. **What is Materialized and Generic views/Logical Views in Bigquery? What is the use of both of them?**

In Google BigQuery, Materialized Views and Logical Views (also called Generic Views) are two types of views used to simplify querying and improve performance, but they serve different purposes and behave differently.

🔍 **1. Logical Views (Generic Views)**
✅ **What is it?**
A Logical View is a virtual table defined by a SQL query. It does not store data physically, but rather runs the underlying SQL query every time the view is accessed.
🧠 Example:
sql
CopyEdit
CREATE VIEW project.dataset.employee_view AS
SELECT name, salary FROM project.dataset.employee WHERE department = 'IT';

📌 **Use Case:**

- Reuse of complex SQL logic.
- Data abstraction layer over raw tables.
- Enforce security via column/row-level access control.

❌ **Drawbacks:**

- No performance gain; always runs the SQL query.
- Slower for large datasets.

💾 **2. Materialized Views (it is a precomputed results stored for performace optimization)**

Method	Description

✅ **What is it?**
A Materialized View is a precomputed version of a query. It stores the results physically and automatically refreshes when the base tables change (incremental updates if possible).
🧠 Example:
sql
CopyEdit
CREATE MATERIALIZED VIEW project.dataset.sales_summary_mv
AS SELECT region, SUM(amount) AS total_sales
FROM project.dataset.sales
GROUP BY region;
📌 Use Case:

- Frequent use of aggregations (e.g., SUM, COUNT, AVG).
- Significantly improves performance and reduces query cost.
- Best for dashboards or repeated reporting queries.

✅ **Benefits:**

- Faster query execution.
- Lower query cost (uses precomputed data).
- Auto-refresh capability.

❌ **Limitations:**

- Only supports SELECT queries with certain aggregations and filters.
- Cannot have JOIN, WINDOW functions, or complex subqueries.
- Limited flexibility compared to logical views.

11. What are slots in Biquery?

What are Slots in BigQuery?

- A slot is a virtual unit of compute used by BigQuery to run queries, load jobs, and export jobs.
- When you run a query in BigQuery, it is divided into smaller stages, and each stage is processed using one or more slots.

✅ **Why Are Slots Important?**

- They **determine the speed** of query execution.
- More slots = **faster query performance**, especially for large datasets.
- Slots are **shared across all queries** in your project (unless you're using reservations).

12. What are the different types of file formats supported in Bigquery?

✅ Supported File Formats in BigQuery

File Format	Description	Use Cases
CSV	Comma-separated values. Simple text format.	Common for exporting/importing data from spreadsheets or databases.

File Format	Description	Use Cases
JSON (newline-delimited)	Each line is a separate JSON object.	Good for semi-structured or nested data. Often used in web apps or NoSQL systems.
AVRO	Binary row-based format with schema support.	Ideal for data exchange between services. Retains data types and schema.
Parquet	Columnar binary file format. Efficient for analytics.	Best for querying large datasets with specific columns.
ORC	Optimized columnar format, mostly used with Hive.	Used for high-performance data storage and queries.
Google Sheets	Data from Google Sheets directly.	Convenient for business users and small datasets.
Datastore Backups	Google Cloud Datastore export format.	Used for importing Datastore backup data.
Bigtable	Used for federated queries from Bigtable.	Good for time-series or wide-column NoSQL data.
LOG/TSV/Custom-delimited text	Via CSV with custom options.	When using non-standard delimiters or log formats.

🔄 When Loading Data into BigQuery:
You can load from:

- Google Cloud Storage (gs://)
- Local file (via bq CLI)
- Google Sheets

🔎 When Querying External Data (Federated Queries):
BigQuery can query external data formats without loading into BQ:

- CSV
- JSON
- AVRO
- Parquet
- ORC
- Google Sheets
- Bigtable

📤 When Exporting Data from BigQuery:

Supported export formats:

- CSV
- JSON (newline-delimited)
- AVRO
- Parquet

13. What is the purpose of using Bigquery?

The purpose of using BigQuery is to enable fast, scalable, and cost-effective analysis of large datasets using SQL in a serverless, fully managed environment. Here's a breakdown of why BigQuery is used:

✅ **Key Purposes of BigQuery**

Purpose	Description
1. Big Data Analytics	Designed for analyzing terabytes to petabytes of data quickly.
2. Serverless Architecture	No infrastructure to manage—Google handles provisioning, scaling, and performance tuning.
3. SQL Interface	Analysts can use standard SQL to query big data efficiently.
4. Real-Time Analytics	Supports streaming data for near real-time insights.
5. High Performance	Uses columnar storage, distributed architecture, and Dremel engine to execute queries in seconds.
6. Cost-Effective	Pay-as-you-go pricing model: only pay for the data you query or store.
7. Integrated with GCP	Easily connects with GCS, Dataflow, Pub/Sub, Looker Studio, and more.
8. Machine Learning Integration	Supports in-database ML with BigQuery ML.
9. Secure & Compliant	Includes features for encryption, IAM, and compliance with industry standards.
10. Data Lake + Warehouse (Lakehouse)	Can analyze structured and semi-structured data (JSON, Avro, Parquet) directly from Cloud Storage.

14. How to edit schema in Bigquery? Can you add columns in while adding the Schema?

In Google BigQuery, you can edit the schema of a table in multiple ways depending on the task (e.g., adding columns, changing data types, etc.).

✅ **1. Can you add columns while adding the schema?**

Yes, you can add new columns when creating a table by defining the schema at that time.

Example using Web UI:

1. Go to BigQuery in the GCP Console.
2. Click on your dataset.

3. Click Create Table.
4. Choose the source (e.g., CSV, JSON, or Empty Table).
5. Under the Schema section, click + Add field.
6. Provide:
    - Name (column name)
    - Type (e.g., STRING, INTEGER)
    - Mode (NULLABLE, REQUIRED, REPEATED)

You can add multiple columns here before creating the table.

✅ **2. How to add columns to an existing table?**

BigQuery allows adding new columns to existing tables. You can do this using:

a) SQL DDL (Data Definition Language):

ALTER TABLE dataset_name. table_name

ADD COLUMN new_column_name DATA_TYPE;

👉 Example:

ALTER TABLE my_dataset.employee

ADD COLUMN department STRING;

You can also add multiple columns:

ALTER TABLE my_dataset.employee

ADD COLUMN (joining_date DATE, is_active BOOLEAN);

b) Web UI:

1. Go to BigQuery > your dataset > select your table.
2. Click Schema tab.
3. Click + Add field.
4. Enter column name, type, and mode.
5. Click Save.

15.What is the difference between REQUIRED and NULLABLE in Bigquery?

✅ **1. NULLABLE**

- Meaning: The field is optional.
- Allows NULL values: ✅ Yes
- Use case: When you are not sure if data will always be available for this column.

✅ **2. REQUIRED**

- **Meaning:** The field is mandatory.
- **Allows NULL values:** ❌ No
- **Use case:** When every row **must** have a value for that column.

📌 **Summary Table**

Feature	REQUIRED	NULLABLE
NULLs allowed	❌ No	✅ Yes
Must provide value	✅ Yes (Mandatory)	❌ No (Optional)
Common use	IDs, essential fields	Optional data fields

16.How to transfer data from GCS to Bigquery (How many ways are there to do this?)

Transferring data from Google Cloud Storage (GCS) to BigQuery can be done in multiple ways, depending on your use case, data volume, and automation needs.

✅ Top Ways to Transfer Data from GCS to BigQuery:

Method	Description	Use Case
1. Manual Load via BigQuery Console	Upload data manually from GCS through the UI.	One-time or small file loads.
2. bq Command-Line Tool	Use the bq load command to load data from GCS.	Scripting, automation via shell scripts.
3. BigQuery Client Libraries (Python, Java, etc.)	Use code to automate loads using BigQuery APIs.	Programmatic or scheduled data loads.
4. Scheduled Queries / Cloud Scheduler + Cloud Functions	Set up automated jobs to load data periodically.	Automated pipeline for recurring loads.
5. BigQuery Data Transfer Service (DTS)	Supports automatic and scheduled imports from GCS.	For recurring, scheduled loads.
6. Cloud Dataflow (Apache Beam)	Stream or batch data from GCS to BigQuery.	Complex transformation before loading.
7. Cloud Composer (Airflow)	Orchestrate GCS → BigQuery loads as part of DAGs.	Complex pipelines with dependencies.
8. Third-party ETL Tools (e.g., Informatica, Talend)	GUI-based ETL tools that support GCS and BigQuery.	Enterprises preferring GUI-based pipelines.

Method	Description	Use Case
9. Auto-ingestion via Event-driven Load	Trigger load when a file is uploaded using Cloud Functions + BigQuery API.	Real-time or near real-time ingestion.

17. How to apply access/restrictions to Big query Tables?

In **Google BigQuery**, you can apply **access control (restrictions or permissions)** at multiple levels:

🔐 **1. Types of Access Control Levels in BigQuery**

Level	Description
Project Level	Grants broad access to all datasets and resources
Dataset Level	Controls access to all tables and views within a dataset
Table/View Level	Controls access to specific tables or views
Column Level	Restrict access to specific columns (using policy tags)

✅ **2. Ways to Apply Access to BigQuery Tables**

🧩 **A. Using Google Cloud Console (UI)**

1. Go to **BigQuery** in the GCP Console.
2. Navigate to the **dataset** or **table**.
3. Click on the **"SHARE"** button (visible for datasets).
4. Add **Principals (users/service accounts)** and assign **roles**.
    - Example Roles:
        - roles/bigquery.dataViewer (Read access)
        - roles/bigquery.dataEditor (Write access)
        - roles/bigquery.dataOwner (Full access)

For **table-level permissions**, you must use the **bq command-line tool** or **IAM policies via API**.

🧩 **B. Using bq Command-Line Tool**

bash

CopyEdit

bq show --format=prettyjson project_id:dataset_id.table_id

To set IAM policy for a table:

bash

CopyEdit

```
bq set-iam-policy project_id:dataset_id.table_id policy.json
```

Where policy.json looks like:

json

CopyEdit

```
{
 "bindings": [
  {
   "role": "roles/bigquery.dataViewer",
   "members": [
    "user:someone@example.com"
   ]
  }
 ]
}
```

🧩 C. Using GCP IAM Policies (via API)

Use **projects.datasets.tables.setIamPolicy** method for programmatic control.

🔍 3. Column-Level Security (Fine-Grained Access)

BigQuery supports **column-level access control** using **Data Catalog Policy Tags**.

Steps:

1. Create **taxonomy and policy tags** in **Data Catalog**.
2. Assign **policy tags** to sensitive columns.
3. Grant roles like roles/datacatalog.tagViewer or roles/datacatalog.tagBindingViewer to users.

📋 Example Use Cases

Scenario	Solution
Only allow analysts to query data	Assign roles/bigquery.dataViewer
Restrict PII columns (e.g., SSN)	Use **Policy Tags** for column-level control
Grant access to one table only	Use **table-level IAM policy**
Remove access from a user	Update IAM policy to revoke user's role

18. How to create tables in Bigquery and what are the different ways to do it?

In BigQuery, you can create tables in multiple ways depending on your use case (manual, automated, or scripted). Here's a complete breakdown:

✅ 1. **Using the Google Cloud Console (UI)**

Steps:

1. Go to BigQuery in the Google Cloud Console.
2. Navigate to your project and dataset.
3. Click "Create Table".
4. Choose source (optional):
    - Empty Table (manual schema)
    - Upload from file (CSV, JSON, Avro, Parquet, etc.)
    - Google Cloud Storage, Google Drive, or other source
5. Provide table name, schema, partition, and cluster details.
6. Click Create Table.

✅ **2. Using SQL (DDL Statements)**

sql

CopyEdit

-- Basic table creation

CREATE TABLE dataset_name.table_name (

column1 STRING,

column2 INT64,

column3 DATE

);

```
-- With partitioning and clustering
CREATE TABLE dataset_name. table_name (
  user_id STRING,
  activity_date DATE,
  event_type STRING
)
PARTITION BY activity_date
CLUSTER BY user_id;
```

✅ **3. Using bq Command-Line Tool**

```
bash
CopyEdit
# Create empty table
bq mk --table project_id:dataset.table_name column1:STRING,column2:INTEGER

# From schema file
bq mk --table --schema=./schema.json dataset.table_name
```

✅ **4. Using Python Client Library**

```
python
CopyEdit
from google.cloud import bigquery
client = bigquery.Client()
schema = [
    bigquery.SchemaField("name", "STRING"),
    bigquery.SchemaField("age", "INTEGER"),
]

table_id = "project_id.dataset.table_name"
table = bigquery.Table(table_id, schema=schema)
```

```
table = client.create_table(table)
print(f"Created table {table. table_id}")
```

✅ **5. From Existing Table (Cloning)**

```
sql
CREATE TABLE dataset.new_table
AS
SELECT * FROM dataset.existing_table;
```

✅ **6. Using BigQuery UI Upload**

- You can upload a file (CSV, JSON, Avro, etc.) directly in the UI.
- Define schema manually or auto-detect.

✅ **7. Using BigQuery Data Transfer Service (BQ DTS)**

- Schedule and automate data import from GCS, Google Ads, YouTube, etc.
- Tables get created automatically during transfer.

✅ **8. Using dbt or Dataform (for Data Engineers)**

- Define table models in .sql files.
- Tables are created when dbt runs.

🔍 Summary Table

Method	Tool/Platform	Notes
UI	Cloud Console	Manual & guided
SQL DDL	SQL Editor	Flexible & powerful
bq CLI	Terminal	Scriptable
Python API	Programmatic	Good for automation
Upload	UI	One-time loads

Method	Tool/Platform	Notes
From Table	SQL	Clone or derive data
BQ DTS	Managed service	Scheduled, automated
dbt/Dataform	CI/CD & modeling	Modern data engineering

19.How to give access on Bigquery Tables/Datasets/Views?

✅ **To Dataset:**

- **Go to BigQuery Console > Dataset > Share Dataset**
- Click **"+ Add Principal"**
- Enter user/group/service account email
- Assign role (e.g., BigQuery Data Viewer, Data Editor, or Data Owner)

✅ **To Table/View:**

- Go to **Table/View > Permissions tab > + Add Principal**
- Assign role (e.g., BigQuery Data Viewer for read access)

✅ **For Authorized Views (Restricted Column Sharing):**

- Create a View
- Add the view as an **authorized view** in the source dataset
- Share the dataset with the user/group

Roles used:

- roles/bigquery.dataViewer – Read
- roles/bigquery.dataEditor – Read/Write
- roles/bigquery.dataOwner – Full control

20. How to load a CSV file in Bigquery using BQ load Command.

Method Tool/Platform Notes

To load a CSV file into BigQuery using the bq load command (from the bq command-line tool), follow this syntax:

✅ Syntax:

bash

CopyEdit

bq load \

--source_format=CSV \

--skip_leading_rows=1 \

dataset_name.table_name \

path_to_file.csv \

schema

🗒️ Example:

Let's say:

- Your CSV file: data.csv
- GCS path: gs://my-bucket/data.csv
- Dataset: sales_data
- Table: orders
- Schema: order_id:INTEGER,customer_name:STRING,amount:FLOAT

bash

bq load \

--source_format=CSV \

--skip_leading_rows=1 \

sales_data.orders \

gs://my-bucket/data.csv \

order_id: INTEGER,customer_name:STRING,amount:FLOAT

⚙️ Common Options:

Method		Tool/Platform	Notes
Option	Description		
--source_format=CSV	Tells BigQuery the file format		
--skip_leading_rows=1	Skips the header row		
--autodetect	Optional, detects schema automatically		
--replace	Optional, replaces the table if it exists		
--noreplace	Prevents replacing existing table		

✅ With Auto Schema Detection:

bash

CopyEdit

bq load \

--source_format=CSV \

--autodetect \

--skip_leading_rows=1 \

sales_data.orders \

gs://my-bucket/data.csv

21. What is the difference between Bigtable and Bigquery?

Here's a **simple comparison** between **Bigtable** and **BigQuery**, two powerful but very different services offered by Google Cloud:

Feature	**Bigtable**	**BigQuery**
Type	NoSQL wide-column database	Serverless data warehouse
Best For	Real-time analytics, time-series data, IoT, monitoring	OLAP, large-scale analytical queries
Use Case	Fast reads/writes for single rows	SQL-based analytics over huge datasets
Data Model	Schema-less, key-value with column families	Relational (tables, rows, columns)
Query Language	No native SQL support (uses API or client libraries)	SQL (standard SQL)

Feature	Bigtable	BigQuery
Storage	Distributed low-latency storage	Columnar storage optimized for analytics
Performance	Millisecond-level latency	Seconds to minutes for complex queries
Scaling	Horizontally scales to petabytes	Serverless, scales automatically
Data Size	Handles large volumes of small data	Handles large volumes of structured data
Examples	Sensor data, gaming leaderboards	Business reporting, marketing analytics

Summary:

- **Bigtable**: Great for high-throughput, low-latency **real-time reads and writes**. Think **NoSQL for operational workloads**.
- **BigQuery**: Designed for **analyzing large datasets** with **SQL**. Ideal for **batch analytics and reporting**.

22. What is the maximum number of Partitions we can create in Bigquery?

In **BigQuery**, the **maximum number of partitions** you can create in a table depends on the **partitioning type**:

✅ **For Date/Datetime/Timestamp Partitioned Tables:**

- **Maximum partitions**: **4,000 partitions per table**

✅ **For Integer-Range Partitioned Tables:**

- **Maximum partitions**: **4,000 partitions per table**

✅ **For Ingestion-Time Partitioned Tables:**

- Also limited to **4,000 partitions**

Additional Notes:

- You can use **clustering** within each partition to further optimize performance.
- If you exceed the 4,000-partition limit, BigQuery will return an error during query or table creation.

23. What is the approach when we reached the maximum number of Partition?

In **BigQuery**, the **maximum number of partitions** allowed **per table** is **4,000 partitions** (as of current limits for ingestion-time and column-based partitioning).

🔍 **When You Reach the Maximum Number of Partitions (4,000):**

Here are the **practical approaches** to handle or avoid this limit:

✅ **1. Use Partition + Clustering**

- **Strategy:** Combine partitioning with clustering.
- **How it helps:** Reduces the need for excessive partitions by improving query performance within each partition.
- **Example:**

sql

CopyEdit

PARTITION BY DATE(event date)

CLUSTER BY user_id, country

✅ **2. Split Data into Multiple Partitioned Tables**

- **Strategy:** Instead of one huge partitioned table, use **table sharding**—split data across multiple partitioned tables.
- **Naming pattern:** events_2023, events_2024, etc.
- **Use wildcard queries** to access all of them together:

sql

CopyEdit

SELECT * FROM `project.dataset.events_*` WHERE _TABLE_SUFFIX BETWEEN '2023' AND '2024'

✅ **3. Reevaluate Partition Key**

- **Strategy:** Use a **coarser granularity** if possible.
- **Example:** If you're using TIMESTAMP or DATE as partition key with **daily granularity**, try switching to **monthly**:

sql

CopyEdit

PARTITION BY DATE_TRUNC(event_date, MONTH)

✅ **4. Use Non-Partitioned Tables with Clustering**

- **Strategy:** If partitioning can't be optimized further, you can consider **just using clustering**.
- **Clustering** doesn't have a strict limit like partitions, and still helps with query performance on large datasets.

✅ **5. Archive Old Data**

- **Strategy:** Move old partitions to long-term storage or export to GCS and delete from BigQuery to stay under the limit.
- Use scheduled queries + data retention policies.

✅ **6. Consider Using Time-based Sharding Instead of Partitioning**

- **Strategy:** For very large time-series datasets, break tables **per month** or **per year** manually and manage access/queries via scripts.

🧠 **Summary Table**

Approach	Purpose	Use When
Partition + Clustering	Improve query efficiency	When partitions < 4,000 but still large
Multiple partitioned tables	Bypass 4,000-partition limit	You're hitting the limit per table
Coarser partitioning (e.g., month)	Reduce number of partitions	Daily partitioning is excessive
Use clustering only	Avoid partition limits	Partitioning isn't effective anymore
Archive old data	Free up partition space	Old data is rarely queried
Time-based sharding	Manual control over large datasets	Partitioning limit cannot be avoided otherwise

24. Which data type is not supported in Partitioning?

In BigQuery, the BIGNUMERIC data type is not supported for partitioning.

✅ Supported Data Types for Partitioning:

- DATE (for Date partitioned tables)
- TIMESTAMP / DATETIME (for Timestamp partitioned tables)
- INTEGER (for Integer-range partitioned tables)

❌ Not Supported:

- BIGNUMERIC
- FLOAT64
- STRING
- BOOLEAN
- GEOGRAPHY
- ARRAY, STRUCT, etc.

❗ Partitioning columns must be of type: DATE, DATETIME, TIMESTAMP, or INTEGER.

25. What is the maximum number of clustering we can create in Bigquery?

In BigQuery, you can define a maximum of 4 clustering columns per table.

📌 Clustering Limits Summary:

- ✅ Maximum clustering columns: 4
- 🧩 Clustering is applied after partitioning (if partitioning is used).
- 🧪 Clustering improves query performance by organizing data based on the values in the specified columns.

Example:

sql

CopyEdit

CREATE TABLE my_dataset.my_table

PARTITION BY DATE (date_column)

CLUSTER BY region, category, user_id, product_id AS

SELECT * FROM source_table;

26. What is Authorised Views in Bigquery?

Authorized Views in **BigQuery** are a security mechanism that allows you to **control access to specific columns or rows** of data without giving users direct access to the underlying base tables.

✅ **Definition:**

An **Authorized View** is a **saved query** (usually a **logical view**) that **grants access to data** in a table, even if the user does **not have direct access** to that table.

🛡️ **Use Case:**

Suppose you have a sensitive table project.dataset.sales_data, and you want to share only specific columns like region, total_sales, but **hide columns like customer_ssn, credit_card_info**. You can:

1. Create a view project.dataset.sales_view with a SELECT on only non-sensitive columns.
2. Authorize that view to access the underlying sales_data table.
3. Grant the external user access only to the view.

🔧 **How to Set Up an Authorized View:**

1. **Create a view**:

sql

CopyEdit

CREATE VIEW dataset.sales_view AS

SELECT region, total_sales

FROM dataset.sales_data;

2. **Authorize the view to access the source table**:
   In the BigQuery console or CLI, authorize the view as a reader of the base table.

bq CLI example:

bash

CopyEdit

bq update --source_table=project.dataset.sales_data --view=project. dataset.sales_view

Or set the authorizedViews in table metadata to include the view.

3. **Grant access to the view**:

bash

CopyEdit

bq add-iam-policy-binding project:dataset.sales_view \

--member='user:someone@example.com' \

--role='roles/bigquery.dataViewer'

✅ **Benefits:**

- Fine-grained **column- and row-level access control**.
- No need to duplicate or export data.
- Centralized **data security and governance**.

🚫 **Important Notes:**

- Authorized views **must be in the same or a different dataset**, but must be **explicitly authorized**.
- Users granted access to the view **cannot bypass it** to access the source table unless separately permitted.

27. What is Time Travel feature in Bigquery?

Time Travel in BigQuery is a powerful feature that allows you to **access the past state of a table** from up to **7 days** in the past (by default).

✅ **Key Points:**

- **Purpose**: To recover deleted or modified data from earlier versions of a table.
- **Default retention**: **7 days** (you can configure it up to 7 days max).
- **Time specification**: You can use a **timestamp** or a **snapshot decorator** (@ syntax).

SELECT *

FROM `project_id. dataset_id.table_id`

FOR **SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP (), INTERVAL 1 HOUR);**

🛠️ **How to Use Time Travel:**

1. Accessing an earlier version of a table:

sql

CopyEdit

SELECT * FROM `project.dataset.table@-3600000`

🔹 This example queries the table **as it was 1 hour (3600000 milliseconds)** ago.

2. Using timestamp:

sql

CopyEdit

SELECT * FROM `project.dataset.table@TIMESTAMP("2025-06-12 10:00:00")`

💡 **Use Cases:**

- Recovering data after accidental DELETE or UPDATE
- Auditing historical records
- Comparing data changes over time

⚠️ **Limitations:**

- Maximum retention is **7 days**.
- Time Travel **doesn't apply to views or external tables**.
- Only works on **native BigQuery tables**, not federated sources.

28. What are snapshots in Bigquery?

Snapshots in BigQuery are point-in-time copies of a table. They capture the **state of a table at a specific moment** without physically duplicating all of its data. This feature is useful for:

- **Data recovery**
- **Auditing**
- **Historical analysis**

🔹 **Key Features of Snapshots:**

Feature	Description
Storage Efficient	Uses **Copy-on-write** technology. Only the changes (deltas) from the original table are stored.
Read-Only	Snapshots are **read-only**; you can query but not modify them.
Cost-Effective	Cheaper than copying the entire table because it reuses the original table's storage.
Time-specific	Created using the exact timestamp from the source table.

🔹 **Syntax to Create a Snapshot**

sql

CopyEdit

```
CREATE SNAPSHOT TABLE project_id.dataset_id.snapshot_table_name
CLONE project_id.dataset_id.source_table
OPTIONS (
  snapshot_time = TIMESTAMP "2024-06-01 10:00:00"
);
```

🔹 **Use Cases**

- 🔁 **Point-in-time recovery**
- 📊 **Audit historical data**

- 🧪 **Testing or validation** without modifying production data
- 📅 **Preserving monthly/quarterly data states**

🔹 **Limitations**

- Cannot update or insert into snapshot tables.
- Not available for views or external tables.
- Only supported in **Enterprise and higher editions**.

29. What is the difference between in both the queries in terms of computation and storage ?
SELECT * FROM EMPLOYEE;

SELECT * FROM EMPLOYEE LIMIT 10;

Query	Computation	Storage Impact	Cost (in BigQuery)
SELECT * FROM EMPLOYEE	Full table scan (all rows, all columns)	None	High (based on table size)
SELECT * FROM EMPLOYEE LIMIT 10	Still full column scan, limit applied after	None	Same as above (not cheaper)

30. If you have 100 columns in a table, how will you query table except 99 columns?

✅ **In BigQuery:**

If you want to select all columns **except one (or a few)**, you can use:

sql

CopyEdit

SELECT * EXCEPT (column_to_exclude)

FROM your_table;

✅ Example:

sql

CopyEdit

SELECT * EXCEPT (col99)

FROM your_dataset. your_table;

But **you cannot exclude 99 columns from 100 in a single EXCEPT** unless you manually list all 99 column names:

sql

CopyEdit

SELECT * EXCEPT (col1, col2, ..., col99)

FROM your_table;

31. How can we implement Row-Level and Column-Level access in Bigquery?

In **BigQuery**, both **Row-Level Security (RLS)** and **Column-Level Security (CLS)** help in restricting access to specific parts of the data. Here's how to implement both:

✅ **1. Row-Level Security (RLS)**

Row-level security restricts access to rows based on conditions, typically user identity or attributes.

Steps to implement RLS:

1. **Create a policy tag condition or use user email matching logic.**
2. **Use CREATE ROW ACCESS POLICY**

🔧 **Example:**

sql

CopyEdit

CREATE ROW ACCESS POLICY sales_region_policy

ON `project.dataset.sales`

GRANT TO ('user:john@example.com')

FILTER USING (region = 'East');

This gives access to only the rows where region = 'East' to the user john@example.com.

📌 **Notes:**

- You can attach **multiple row access policies**.
- Policies use **Boolean conditions** to allow access.

✅ **2. Column-Level Security (CLS)**

Column-level security restricts access to **specific columns** using **Data Catalog Policy Tags**.

Steps to implement CLS:

1. **Create Policy Tags** in **Data Catalog**.
2. **Assign tags to columns** in your BigQuery table schema.
3. **Grant IAM permissions (Fine-grained Reader)** for those policy tags.

🔧 **Example:**

Suppose you have a column salary you want to restrict:

1. In **Data Catalog**, create a policy tag: sensitive. salary
2. Assign this tag to the salary column.
3. Grant access:

bash

CopyEdit

```
gcloud data-catalog policy-tags add-iam-policy-binding \
 --policy-tag="projects/my-project/locations/us/taxonomies/123/policyTags/salary" \
 --member="user:john@example.com" \
 --role="roles/datacatalog.categoryFineGrainedReader"
```

Now only john@example.com can access the salary column.

✅ **Summary Table**

Feature	Tool Used	Access Level	Example Usage
Row-Level Security	CREATE ROW ACCESS POLICY	Row	Restrict by region, department, etc.
Column-Level Security	Data Catalog Policy Tags + IAM	Column	Hide salary, ssn columns from some users

Google Big Query Real time scenario-based questions and answers

✅ **32. How do you optimize a slow-running BigQuery query?**

Answer:

- Use **partitioned tables** to scan less data.
- Use **clustering** to group related rows.

Feature	Tool Used	Access Level	Example Usage

- Avoid SELECT *; query only needed columns.
- Use **materialized views** or **caching** for repeated queries.
- Check the **execution plan** with EXPLAIN.
- Filter early using WHERE clauses.
- Avoid **cross joins** unless necessary.

✅ 33. You have a 10TB CSV in GCS. How will you load it efficiently into BigQuery?

Answer:

- Use **parallel load** with multiple CSV chunks (split the file).
- Use **load jobs** instead of streaming.
- Use **compressed (gzip) files** to reduce cost.
- Use **schema auto-detection** or define the schema explicitly.
- Use the bq load command or Python SDK.

✅ 34. How can you handle schema evolution in BigQuery?

Answer:

- Use ALLOW_FIELD_ADDITION for adding new fields.
- Use ALLOW_FIELD_RELAXATION to change required fields to nullable.
- Use **compatible schemas** during load/merge.
- For complex changes, create a new table version and migrate.

✅ 35. A column stores nested JSON data. How do you query inner fields?

Answer:
Use UNNEST() and dot notation.

sql

CopyEdit

SELECT user.id, user.name

FROM my_dataset.nested_table,

Feature	Tool Used	Access Level	Example Usage
UNNEST (user_info) AS user			

✅ 36. Your job needs to ingest 1 million rows per minute. How do you design it in BigQuery?

Answer:

- Use **streaming inserts** with batching.
- Use **Pub/Sub + Dataflow + BigQuery** for real-time pipelines.
- Use partitioned and clustered tables to improve performance.
- Monitor quotas and apply backoff strategies.

✅ 37. How do you handle deduplication in BigQuery during streaming?

Answer:

- Add a **unique identifier** (UUID) or timestamp to each row.
- Use ROW_NUMBER() over partitions and select latest rows.
- Use MERGE statements for deduplication into a final table.

✅ 38. How do you schedule a daily BigQuery job to aggregate sales data?

Answer:

- Use **Cloud Scheduler** to trigger a **Cloud Function** or **Workflow** that runs a query.
- Or, use **scheduled queries** directly in BigQuery UI.
- Store output in a partitioned table.

✅ 39. You want to reduce BigQuery storage cost. What can you do?

Answer:

- Use **partitioned** and **clustered tables**.
- **Expire** old partitions with partition expiration.
- Delete/archive unused tables.

Feature	Tool Used	Access Level	Example Usage

- Use **compressed formats** like Avro/Parquet.
- Use **table snapshots** instead of full copies.

✅ 40. How do you load data incrementally from GCS to BigQuery?

Answer:

- Load only new files using **naming patterns or timestamps**.
- Use **metadata files** to track last loaded files.
- Use **Dataflow or Cloud Functions** triggered on new GCS uploads.

✅ 41. How do you implement Slowly Changing Dimension (SCD Type 2) in BigQuery?

Answer:

- Use MERGE with WHEN MATCHED THEN UPDATE, WHEN NOT MATCHED THEN INSERT.
- Maintain start_date, end_date, and is_current columns.

✅ 42. How do you check which user ran a specific query in BigQuery?

Answer:

- Use **Cloud Audit Logs** (data_access logs).
- You can filter by resource type: bigquery_resource.

✅ 43. How can you export BigQuery results as CSV to GCS?

Answer:

sql

CopyEdit

```
EXPORT DATA OPTIONS(
  uri='gs://your_bucket/results_*.csv',
  format='CSV',
```

Feature	Tool Used	Access Level	Example Usage

```
 overwrite=true
) AS
SELECT * FROM your_dataset.table_name;
```

✅ 44. How to avoid charges when testing queries?

Answer:

- Use --dry_run flag with bq CLI.
- Estimate scanned bytes with EXPLAIN.
- Use **LIMIT** or smaller partitions during testing.

✅ 45. How do you ensure data quality in a BigQuery pipeline?

Answer:

- Add **data validation rules** using SQL.
- Use **dbt tests** or **custom validation queries**.
- Check for **nulls, duplicates, data type mismatches**.
- Log anomalies to monitoring tools (e.g., Stackdriver).

✅ 46. What's the difference between partitioning by ingestion time vs a column?

Answer:

- **Ingestion-time partitioning** is automatic based on _PARTITIONTIME.
- **Column-based partitioning** uses a user-defined DATE/TIMESTAMP column.
- Column-based is better for **event-based analysis**.

✅ 47. How do you perform real-time dashboarding using BigQuery?

Answer:

- Use **streaming inserts** or Dataflow to ingest live data.

Feature	Tool Used	Access Level	Example Usage

- Use **Looker Studio**, **Tableau**, or **Power BI** for dashboards.
- Minimize latency with **materialized views** or **cached tables**.

✅ 48. How to use BigQuery for log analysis from GCS logs?

Answer:

- Load log files from GCS into a **structured table**.
- Use REGEXP_EXTRACT, PARSE_DATE, etc., to parse logs.
- Build time-series and anomaly detection queries.

✅ 49. You want to join a 10M row table with a 100 row reference table. How?

Answer:

- Use **broadcast join** (BigQuery does this automatically if the small table fits in memory).
- Ensure the small table is **materialized in a subquery** or CTE.

✅ 50. How do you automate BigQuery table archival?

Answer:

- Use **partition expiration** or SET OPTIONS.
- Use a **scheduled query** to copy old data to an archive table.
- Then delete from the original table.

✅ 51. You ran a query and got Resources exceeded during query execution error. How do you fix it?

Answer:

- Split into smaller queries.
- Use **partitioned/clustered tables**.
- Avoid expensive joins or subqueries.
- Increase slot allocation if using **reservations**.

Feature	Tool Used	Access Level	Example Usage

✅ 52. How do you mask PII data in BigQuery?

Answer:

- Use **Dynamic Data Masking** (if available).
- Use **views** with masked expressions (e.g., SUBSTR, REGEXP_REPLACE).
- Store masked and unmasked data in separate access-controlled tables.

✅ 53. How do you backfill data for the last 6 months in a partitioned table?

Answer:

- Use a **loop in a script** or a job with bq CLI to run per-day queries.
- Insert or overwrite into partitioned table with WHERE event_date = CURRENT_DATE - INTERVAL X DAY.

✅ 54. How do you version control BigQuery SQL and pipelines?

Answer:

- Store SQL queries and dbt models in **Git**.
- Use **CI/CD** pipelines (Cloud Build, GitHub Actions) to deploy.

✅ 55. How do you load Avro files with nested data into BigQuery?

Answer:

- Avro supports nested schemas; BigQuery automatically detects them.
- Use:

bash

CopyEdit

bq load --source_format=AVRO dataset.table gs://bucket/file.avro

---

✅ 56. How do you audit who accessed a BigQuery table and when?

Feature	Tool Used	Access Level	Example Usage

Answer:

- Use **Cloud Audit Logs** → Admin Activity & Data Access logs.
- Filter logs by resource type: bigquery_table.
- Use gcloud logging read or query logs in **Log Explorer**.

✅ **57. How can you monitor BigQuery costs for specific users or queries?**

Answer:

- Use **BigQuery INFORMATION_SCHEMA.JOBS_BY_PROJECT** or **JOBS_BY_USER**.
- Enable **Cloud Billing export to BigQuery**.
- Filter by user_email, job_type, total_bytes_billed.

✅ **58. How do you handle skewed partitions in BigQuery?**

Answer:

- Combine **partitioning** with **clustering**.
- Re-partition data based on distribution.
- Use **sharded tables** or balance ingestion.

✅ **59. Can BigQuery join external tables from Cloud Storage?**

Answer:

- Yes, using **external tables**.
- Example:

```
sql

CopyEdit

CREATE OR REPLACE EXTERNAL TABLE my_ext_table

OPTIONS (

 format = 'CSV',

 uris = ['gs://my-bucket/data/*.csv']

);
```

Feature	Tool Used	Access Level	Example Usage

SELECT * FROM my_ext_table JOIN dataset.internal_table USING(id);

✅ 60. How do you delete data from a partitioned BigQuery table?

Answer:
Use a DELETE with a partition filter:

sql

CopyEdit

DELETE FROM dataset.table

WHERE _PARTITIONDATE = '2024-05-01' AND status = 'obsolete';

✅ 61. What are materialized views and when should you use them?

Answer:

- Precomputed query results stored in BigQuery.
- Automatically refreshed.
- Useful when:
    - Query results don’t change often.
    - You want **fast** access to aggregated data.
    - Reduce compute costs on repeated complex queries.

✅ 62. A table is growing rapidly daily. How do you manage long-term storage and querying efficiency?

Answer:

- Use **partition expiration** to automatically delete old partitions.
- Archive old data to **separate tables or GCS**.
- Implement **views or union of recent + archive tables** for access.

✅ 63. A user is querying a large unpartitioned table and experiencing slow performance. What's your solution?

Answer:

- Convert the table into a **partitioned table**:

sql

Feature	Tool Used	Access Level	Example Usage

CopyEdit

CREATE TABLE dataset.partitioned_table

PARTITION BY DATE(created_at)

AS SELECT * FROM dataset.unpartitioned_table;

- This reduces scanned data dramatically.

✅ 64. How do you handle schema mismatches when loading from GCS into BigQuery?

Answer:

- Use the --schema_update_option=ALLOW_FIELD_ADDITION or ALLOW_FIELD_RELAXATION
- Validate schemas using bq show or programmatically check JSON schema.

✅ 65. A user wants to export a 500 GB BigQuery table to GCS. How do you do it efficiently?

Answer:

- Use EXPORT DATA with wildcard URIs:

sql

CopyEdit

EXPORT DATA OPTIONS (

uri = 'gs://bucket_name/export/data-*.parquet',

format = 'PARQUET',

overwrite = true

)

AS SELECT * FROM dataset.large_table;

- Exporting in **Parquet/Avro** is more efficient for large exports

✅ 66. You need to transfer data from AWS S3 to BigQuery. What's your approach?

Answer:

1. Transfer S3 → GCS using **Storage Transfer Service** or gsutil.
2. Load data from GCS to BigQuery using bq load or scheduled queries.

Feature	Tool Used	Access Level	Example Usage

✅ 67.A downstream team needs data every 10 minutes. How do you handle this in BigQuery?

Answer:

- Use **streaming insert** or **Pub/Sub → Dataflow → BigQuery**.
- Create **materialized views** or **intermediate tables** refreshed every 10 minutes using **Cloud Scheduler** or **scheduled queries**.

✅ 68. How do you backfill missing partitions in a partitioned table?

Answer:

- Write a **script** or use bq CLI to insert data per missing partition using a loop:

bash

CopyEdit

```
for i in {0..6}; do
  bq query --use_legacy_sql=false \
  "INSERT INTO dataset.table
   SELECT * FROM source_table
   WHERE event_date = DATE_SUB(CURRENT_DATE(), INTERVAL $i DAY);"
done
```

✅ 69. A user wants to export a 500 GB BigQuery table to GCS. How do you do it efficiently?

Answer:

- Use EXPORT DATA with wildcard URIs:

sql

CopyEdit

```
EXPORT DATA OPTIONS (
  uri = 'gs://bucket_name/export/data-*.parquet',
  format = 'PARQUET',
  overwrite = true
```

Feature	Tool Used	Access Level	Example Usage

)

AS SELECT * FROM dataset.large_table;

- Exporting in **Parquet/Avro** is more efficient for large exports.

✅ 70. You are asked to audit changes in a table over time. What's your approach?

Answer:

- Implement **Change Data Capture (CDC)** using timestamps or audit fields.
- Use **BigQuery table snapshots** or **versioned records**.
- Example:

sql

CopyEdit

SELECT * FROM dataset.my_table FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 DAY);

✅ 71. You want to join a BigQuery table with a GCS file daily. What's your approach?

Answer:

- Create an **external table** for GCS file.
- Use a scheduled query to join external and internal tables and store results in a new table.

✅ 72. You need to remove a specific row from a partitioned table. What's the best way?

Answer:

- Use DELETE with partition filter:

sql

CopyEdit

DELETE FROM dataset.events

WHERE _PARTITIONTIME = TIMESTAMP ('2025-06-01') AND event_id = 'abc123';

Feature	Tool Used	Access Level	Example Usage

✅ 73. How do you minimize cost when running exploratory queries on a huge table?

Answer:

- Use:
    - **Preview table** (no cost)
    - LIMIT, partition filters
    - SELECT column instead of SELECT *
    - Temporary tables
    - DRY RUN to estimate cost

✅ 74. A Data Analyst accidentally deleted a BigQuery table. Can you recover it?

Answer:

- If **table expiration** or **Time Travel** is enabled, restore:

sql

CopyEdit

CREATE OR REPLACE TABLE dataset.table

AS SELECT * FROM dataset.table FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR);

- Otherwise, check if table was exported to GCS for recovery.

✅ 75. Scenario: Optimize Slow BigQuery Query

Q: Your Big Query query is running slowly. How do you identify the bottleneck and improve performance?

A:

- Use EXPLAIN or Query Execution Details tab to analyze stage breakdown.
- Common fixes:
    - Avoid SELECT *; only fetch required columns.
    - Partition large tables using a DATE or TIMESTAMP field.

Feature	Tool Used	Access Level	Example Usage

- Use clustering on frequently filtered columns.
- Materialize intermediate results using temporary tables or materialized views.

✅ 76. Scenario: You have multiple CSV files in a GCS bucket and want to load them into a BigQuery table. How would you do it efficiently?

Answer:

- Use a **wildcard URI** like gs://my-bucket/data_*.csv to load multiple files at once.
- Use the bq load CLI or LOAD DATA SQL command with the correct schema and file format.
- Ensure:
  - Schema is consistent across files.
  - File format options like skip_leading_rows, field_delimiter are properly set.

✅ 77. Scenario: You want to update a column value for specific records in a BigQuery table. What is the best way?

- Answer:
  Use the MERGE or UPDATE SQL statement.
- UPDATE my_dataset.my_table
- SET status = 'active'
- WHERE last_login > DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)

✅ 78. Scenario: Daily Partitioned Table with Overwrite
Q: You want to load daily data into a partitioned BigQuery table. Each day's data should overwrite only that day's partition. How?
A:
Use partition decorators or WRITE_TRUNCATE on the partition:
sql
CopyEdit
INSERT INTO dataset.table_name PARTITION (DATE(_PARTITIONTIME))
SELECT * FROM staging_table
WHERE date = '2025-06-13';

Feature	Tool Used	Access Level	Example Usage

Or use:
bash
CopyEdit
bq load --autodetect \
--source_format=CSV \
--replace \
dataset.table_name$20250613 gs://bucket/data.csv

✅ **79. Scenario: Your query performance is slow on a large table. How do you troubleshoot?**
Answer:
Steps:

1. **Check Execution Details** in the Query Plan tab.
2. Optimize using:
    - **Partitioning** (e.g., by date)
    - **Clustering** (e.g., by user_id)
3. **Avoid SELECT ***.
4. **Materialize common subqueries** with temporary tables or views.
5. Use **table preview** to inspect sample data instead of querying large datasets.

✅ **80. Scenario: How do you automate a daily data load from GCS to BigQuery?**
Answer:
Use **Cloud Composer (Airflow)** or **Cloud Functions + Scheduler**:

- In Composer, define a DAG with a GCSToBigQueryOperator.
- Set up a daily schedule and monitor success/failure using Airflow UI.

python
CopyEdit
GCSToBigQueryOperator(
  task_id='load_daily_data',
  bucket='my-bucket',
  source_objects=['data/daily/*.csv'],
  destination_project_dataset_table='project.dataset.table',
  source_format='CSV',
  write_disposition='WRITE_APPEND',
  skip_leading_rows=1,
)

✅ **81. Scenario: Your BigQuery table is growing too large over time. How do you manage cost?**
Answer:

Feature	Tool Used	Access Level	Example Usage

- Implement **table partitioning** by DATE or TIMESTAMP.
- Use **partition expiration**:
  sql
  CopyEdit
  CREATE TABLE project.dataset.table (
  ...
)
  PARTITION BY DATE(event_time)
  OPTIONS (
  partition_expiration_days = 30
)
- Consider **table clustering** and **materialized views** to reduce repeated query costs.
- Archive old data into GCS and delete from BigQuery if needed.

✅ **82. Scenario: You want to join a 1TB table with a 10MB table. What is the optimal join strategy?**
Answer:

- Use a **broadcast (map-side) join**.
- Ensure the smaller table is the **right-hand side** of the join.
- Use a **JOIN hint** if needed:
  sql
  CopyEdit
  SELECT /*+ BROADCAST(tiny_table) */ *
  FROM big_table
  JOIN tiny_table
  ON big_table.id = tiny_table.id

✅ **83. Scenario: How do you ensure schema consistency when loading data into BigQuery?**
Answer:

- Define **explicit schema** rather than using autodetect.
- Use **schema validation tools** in ETL pipeline.
- Enable **schema enforcement** in data ingestion pipeline.
- Validate data using **Dataform or dbt** before loading.

✅ **84. Scenario: You want to implement incremental load in BigQuery from a source system. How would you do it?**
Answer:

- Identify a **watermark column** (e.g., last_modified).
- Use this in the WHERE clause for new/changed records.
  sql
  CopyEdit
  SELECT * FROM source_table

Feature	Tool Used	Access Level	Example Usage

WHERE last_modified > (SELECT MAX (last_modified) FROM target_table)

- Use MERGE statement to perform upserts.

✅ **85. Scenario: You need to generate a report on BigQuery with aggregated metrics, but refresh it every hour. What's the efficient way?**
Answer:

- Use **Materialized Views** if possible:
  sql
  CopyEdit
  CREATE MATERIALIZED VIEW my_dataset.sales_mv
  AS SELECT region, SUM(sales) AS total_sales
  FROM my_dataset.sales
  GROUP BY region;
- Refreshes automatically with change detection.
- Use **scheduled queries** if more complex logic is needed.

✅ **86. Scenario: You need to mask sensitive data like email IDs before sharing query results. What is your approach?**
Answer:

- Use **STRING manipulation** to mask values.
  sql
  CopyEdit
  SELECT
  REGEXP_REPLACE(email, r'(^[^@]+)', 'xxxxx') AS masked_email
  FROM users;
- Or use **BigQuery Data Masking Policies** (if using **BigQuery Column-Level Security**).

✅ **87. Scenario: You want to overwrite only a partition of a BigQuery table during load. How do you achieve it?**
Answer:
Use **partition decorators** in the destination table.
bash
CopyEdit
bq load \
--source_format=CSV \
my_dataset.my_table$20250614 \
gs://my-bucket/data_20250614.csv \
schema.json
This loads data **only into the 2025-06-14 partition**, replacing it.

Feature	Tool Used	Access Level	Example Usage

✅ **88. Scenario: You want to implement SCD Type 2 in BigQuery. How would you do that?**
Answer:

- Maintain effective_start_date, effective_end_date, and is_current fields.
- Use a **MERGE** strategy:
  - Expire old records.
  - Insert new ones if changed.

```
sql
CopyEdit
MERGE target_table T
USING source_table S
ON T.id = S.id AND T.is_current = TRUE AND T.hash = S.hash
WHEN MATCHED THEN
  UPDATE SET is_current = FALSE, effective_end_date =
CURRENT_DATE()
WHEN NOT MATCHED THEN
  INSERT (id, name, hash, effective_start_date, is_current)
  VALUES (S.id, S.name, S.hash, CURRENT_DATE(), TRUE)
```

✅ **89. Scenario: How do you query only the last 7 days of data from a partitioned table?**
Answer:
Assuming partitioned on event_date:

```
sql
CopyEdit
SELECT * FROM my_table
WHERE event_date BETWEEN DATE_SUB(CURRENT_DATE(),
INTERVAL 7 DAY) AND CURRENT_DATE()
```

This ensures partition pruning, improving performance.

✅ **90. Scenario: How do you secure specific columns (e.g., salary) in BigQuery from unauthorized access?**
Answer:

- Use **Column-Level Security**:
  - Define **policy tags** using **Data Catalog**.
  - Apply access controls on tags.
  - Assign IAM roles like bigquery.datapolicyUser.

```
sql
CopyEdit
ALTER TABLE my_table
ALTER COLUMN salary
SET OPTIONS (policy_tags =
['projects/my-project/locations/us/taxonomy/123456/policyT
ags/abcdef'])
```

Feature	Tool Used	Access Level	Example Usage

✅ **91. Scenario: Your query has a CROSS JOIN but returns a huge output. How do you fix or limit it?**
Answer:

- Use explicit JOIN conditions.
- Apply **filter conditions before the join**.
- If Cartesian product is intended, apply **LIMIT**:
  sql
  CopyEdit
  SELECT *
  FROM table1, table2
  LIMIT 10000
  Or rewrite to use INNER JOIN with ON condition.

✅ **92. Scenario: You want to perform a rolling 7-day average of sales per product. How would you do it?**
Answer:
sql
CopyEdit
SELECT
  product_id,
  sales_date,
  AVG(sales) OVER (
    PARTITION BY product_id
    ORDER BY sales_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
) AS moving_avg
FROM sales_table
Uses **window functions** for rolling aggregates.

✅ **93. Scenario: How can you share BigQuery datasets securely with another GCP project?**
Answer:

- Share the dataset via **IAM roles** (BigQuery Data Viewer).
- Add the external **project's service account** or **user group**.
- Or use **Authorized Views** or **Row-Level Security** for more control.

✅ **94. Scenario: You are receiving JSON files in GCS. How do you ingest nested data into BigQuery?**
Answer:

- Use --source_format=NEWLINE_DELIMITED_JSON.
- Define nested schema in a schema file or via autodetect.
- Flatten using UNNEST after loading:
  sql
  CopyEdit

Feature	Tool Used	Access Level	Example Usage

```
SELECT
  user.name,
  item.id
FROM my_table
CROSS JOIN UNNEST(user.purchases) AS item
```

✅ **95. Scenario: You want to avoid duplicate data during ingestion from GCS. What's your approach?**
Answer:

- Add a **deduplication key** or hash column (e.g., MD5 of all columns).
- Use MERGE with deduplication logic.
- Or stage data into a temporary table and apply ROW_NUMBER() to filter duplicates.

✅ **96. Scenario: How can you schedule a BigQuery query to run hourly and store results?**
Answer:
Use **Scheduled Queries** in BigQuery UI:

- SQL editor → Schedule → Set frequency.
- Destination table → Choose write mode (append, overwrite).
- Monitor in **Scheduled Queries section**.

✅ **97. Scenario: How do you monitor BigQuery cost at dataset or query level?**
Answer:

- Enable **BigQuery Audit Logs** (Cloud Logging).
- Use *INFORMATION_SCHEMA.JOBS_BY_ views**.
- Create dashboards using **Looker Studio** or **BigQuery usage reports**.

```
sql
CopyEdit
SELECT user_email, total_bytes_billed, query
FROM `region-us`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE creation_time >
TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
```

✅ **98. Scenario: You are asked to give different access levels to different users on the same BigQuery table. What is your solution?**
Answer:

- Use **Row-Level Security (RLS)** or **Authorized Views**.
- Create a filtered view:

```
sql
CopyEdit
```

Feature	Tool Used	Access Level	Example Usage

```
CREATE VIEW secure_view AS
SELECT * FROM orders
WHERE region = SESSION_USER()
```

- Share only the **view** and not the base table.

✅ **99. Scenario: You want to execute a BigQuery query from a Python script. How do you do it?**
Answer (using google-cloud-bigquery):

```
python
CopyEdit
from google.cloud import bigquery

client = bigquery.Client()
query = "SELECT * FROM `project.dataset.table` LIMIT 10"
result = client.query(query)

for row in result:
   print(row)
```

✅ **100. Scenario: How do you backup a BigQuery table daily?**
Answer:

- Use a scheduled query with CREATE TABLE AS SELECT *.
- Or use **export to GCS**:

```
bash
CopyEdit
bq extract --destination_format=CSV my_dataset.my_table
gs://my-bucket/backups/table_$(date +%Y%m%d).csv
```

Automate using Cloud Composer or Scheduler + Cloud Function.

✅ **101. Scenario: Incremental Load Using Last Modified Timestamp**
Q: You are loading new records daily from GCS. How do you ensure only the latest data gets ingested?
A:
Use a last_updated timestamp column and filter based on the latest ingested value (store it in metadata or use MAX (last_updated) from target table).

✅ **102. Scenario: Validate Data Before Loading to BigQuery**
Q: You want to validate schema and nulls in incoming data before loading it into BigQuery. How?

Feature	Tool Used	Access Level	Example Usage

A:

- Load file into staging table.
- Run validation SQL:
  sql
  CopyEdit
  SELECT * FROM staging_table WHERE critical_column IS NULL;
- If validation passes, load into target:
  sql
  CopyEdit
  INSERT INTO target_table SELECT * FROM staging_table WHERE ...

103: Your table is growing rapidly. How would you optimize a BigQuery query on a 5TB table to ensure minimal cost and maximum performance?
Answer:

- Use **partitioning** on time-based fields (e.g., ingestion_time).
- Apply **clustering** on frequently filtered columns (e.g., user_id, region).
- Avoid SELECT *; select only required columns.
- Use **materialized views** for repeated heavy aggregations.
- Monitor with **Query Execution Plan** in the Query UI.

104: How do you load data into a partitioned BigQuery table from a GCS file that includes timestamps?
Answer:
sql
CopyEdit
CREATE TABLE dataset.table_name (
  id INT64,
  event_time TIMESTAMP,
  ...
)
PARTITION BY DATE(event_time);

- Use bq load:
  bash
  CopyEdit
  bq load --source_format=CSV \
  --autodetect \
  dataset.table_name \
  gs://bucket/data.csv
  Ensure the schema contains the event_time column for correct partitioning.

105: You need to implement RLS (Row-Level Security) for a sensitive column in BigQuery. How would you achieve that?

Answer:

- Create **Authorized Views** or **Row Access Policies**.
- Example:
  sql
  CopyEdit
  CREATE ROW ACCESS POLICY regional_policy
  ON dataset.sales_data
  GRANT TO ('user:india_team@example.com')
  FILTER USING (region = "India");

GCS - Real-Time Scenarios

106: Your data pipeline must archive daily processed files. How do you automate this using GCS lifecycle?

Answer:

- Define a **lifecycle policy** in GCS bucket:
  json
  CopyEdit

```
{
 "rule": [
  {
   "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
   "condition": {"age": 30}
  }
 ]
}
```

107: You want to transfer large datasets from AWS S3 to GCS. How do you handle this securely?

Answer:

- Use **Storage Transfer Service**.
- Set up AWS access credentials with **temporary access key**.
- Define a transfer job via:
  bash
  CopyEdit
  gcloud transfer jobs create --source-s3-bucket=source-bucket \
  --destination-gcs-bucket=target-bucket \
  --source-s3-access-key-id=xxx --source-s3-secret-access-key=yyy

---

108: How to ensure only specific users can upload files but not delete in a GCS bucket?

Answer:

- Use **IAM roles** at bucket level.
- Assign roles/storage.objectCreator to allow uploads.
- Avoid granting roles/storage.objectAdmin.

109: You need to rename files in a GCS bucket. GCS doesn't support rename — how would you handle it?
Answer:

- Copy the file with the new name using gsutil cp, then delete the original:
  bash
  CopyEdit
  gsutil cp gs://bucket/old_name.csv gs://bucket/new_name.csv
  gsutil rm gs://bucket/old_name.csv
- Automate with Python if done repeatedly.

110: A GCS bucket is used by multiple teams. How do you control access to different folders inside the bucket?
Answer:

- Use **IAM Conditions**:
  json
  CopyEdit
  {
  "title": "Restrict Folder Access",
  "expression":
  "resource.name.startsWith('projects/_/buckets/my-bucket/objects/folderA/')"
  }
- Apply role at object-level with fine-grained access.

111.Can we create two same bucket name?

No, you cannot create two buckets with the same name in Google Cloud Platform (GCP) — bucket names must be globally unique across all GCP projects and regions.

112.How many GCS buckets we can create in GCP ?

✅ GCS (Google Cloud Storage) Bucket Limits

- Default per project:
  You can create up to 100 buckets per project by default.
- Can it be increased?
  Yes, you can request a quota increase by submitting a request to Google Cloud support.

113.How to create a bucket in GCS using gsutil?

To create a bucket in Google Cloud Storage (GCS) using gsutil, you can use the gsutil mb (make bucket) command.

✅ Syntax:

bash

CopyEdit

gsutil mb -p [PROJECT_ID] -c [STORAGE_CLASS] -l [LOCATION] gs://[BUCKET_NAME]/

✅ Example:

bash

CopyEdit

gsutil mb -p my-gcp-project -c STANDARD -l us-central1 gs://my-unique-bucket-name/

🔍 **Parameters:**

- -p: Your GCP project ID
- -c: Storage class (e.g., STANDARD, NEARLINE, COLDLINE, ARCHIVE)
- -l: Bucket location/region (e.g., us, us-central1, asia-south1)
- gs://my-unique-bucket-name/: The globally unique name of your bucket

114.How to encrypt data in GCS?

In Google Cloud Storage (GCS), data is encrypted at rest by default using Google-managed encryption keys. However, if you want more control, GCS also supports customer-supplied and customer-managed encryption keys.

Here are the three main ways to encrypt data in GCS:

🔐 1. Google-Managed Encryption Keys (Default)

- No action needed from your side.
- Google automatically encrypts your data with AES-256 and manages key rotation and storage.
- Best for general use cases.

🔑 2. Customer-Managed Encryption Keys (CMEK)

- You manage keys using Cloud Key Management Service (KMS).
- Suitable when you need control over key rotation, access, and audit logs**.**

✅ **Steps to use CMEK:**

1. Create a KMS key in Google Cloud KMS:

gcloud kms keys create my-key \

--location=global \

--keyring=my-key-ring \

--purpose=encryption

2. Assign IAM role (Cloud KMS CryptoKey Encrypter/Decrypter) to the GCS service account.
3. Create a bucket with CMEK:

gsutil mb -p [PROJECT_ID] -l us gs://[BUCKET_NAME]/

gsutil bucketprops set -k [KMS_KEY_RESOURCE_PATH] gs://[BUCKET_NAME]

Example KMS key resource path:

projects/my-project/locations/global/keyRings/my-key-ring/cryptoKeys/my-key

---

🔐 **3. Customer-Supplied Encryption Keys (CSEK)**

- You supply and manage the raw encryption keys (base64-encoded).
- GCS does not store your key — you must provide it with every request.

✅ **To upload an encrypted file using CSEK:**

gsutil -o "GSUtil:encryption_key=<base64-encoded-key>" cp file.txt gs://your-bucket/

✅ **To download it:**

gsutil -o "GSUtil:decryption_key=<base64-encoded-key>" cp gs://your-bucket/file.txt .

🔍 **Summary Table:**

Encryption Type	Key Managed By	Key Rotation	Use Case
Google-Managed (Default)	Google	Automatic	Default, low-maintenance
Customer-Managed (CMEK)	You (via Cloud KMS)	Manual	Compliance, audit, key control
Customer-Supplied (CSEK)	You (own key)	Manual	Max control, must manage keys

115.Can we move files directly from Standard Storage Class to Archive Storage in GCS?

Yes, you can move files directly from Standard Storage Class to Archive Storage Class in GCS (Google Cloud Storage).

🔄 **How to move (change storage class):**

You can use any of the following methods:

✅ 1. Using `gsutil rewrite` command:

bash Copy Edit

```
gsutil rewrite -s ARCHIVE gs://your-bucket/your-object
```

This will rewrite the object and change its storage class to **ARCHIVE**.

✅ 2. Using GCS Lifecycle Policy (automated):

You can define a **lifecycle rule** to transition objects after a certain number of days to Archive class.

Example JSON:

json Copy Edit

```
{
  "rule": [
    {
      "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
      "condition": {"age": 30}
    }
  ]
}
```

Apply it with:

bash Copy Edit

```
gsutil lifecycle set lifecycle.json gs://your-bucket
```

✅ 3. Re-upload manually (not recommended unless needed):

Download the file and re-upload with the desired storage class using:

bash

```
gsutil cp -s ARCHIVE gs://source-bucket/file gs://target-bucket/file
```

✅ Important Notes:

- Changing storage class using `rewrite` will **incur a cost** as it rewrites the file.
- If you set lifecycle policies, GCS will transition files **automatically** based on your rules.

116.If we are loading table from gcs to bigquery with data overwrite option ,will default value of column remain same or value will get lost ?

If you're **loading data from GCS to BigQuery with the overwrite option** (using WRITE_TRUNCATE), here's what happens:

✅ **Default values of columns:**

- **WILL NOT be preserved** if the column is **not present** in the data you're loading.
- BigQuery **does not apply schema-level default values** during data load from external sources like GCS.
- So, **default values will be lost** unless they are **explicitly included in the incoming data**.

✅ **Recommendation:**

If you want to preserve default values:

- Either **include the column with values in the incoming data**, or
- **Use a staging table**, then use SQL like INSERT INTO ... SELECT ..., IFNULL(status, 'active') to apply defaults.

117. How to transfer data from AWS to GCS?

To **transfer data from AWS (e.g., S3)** to **Google Cloud Storage (GCS)**, you can use multiple methods depending on the data size, frequency, and available tools. Here are the **most commonly used options**:

✅ 1. Using `gsutil` (Manual or Scripted Transfer)

bash Copy Edit

```
# Step 1: Configure AWS credentials
export AWS_ACCESS_KEY_ID=your_aws_access_key
export AWS_SECRET_ACCESS_KEY=your_aws_secret_key

# Step 2: Copy from S3 to GCS using gsutil
gsutil -m cp "s3://your-source-bucket-name/*" "gs://your-target-gcs-bucket/"
```

> Requires `gsutil` installed and `boto` configured with AWS credentials.

✅ 2. Using GCS Transfer Service (Recommended for large-scale or recurring transfers)

1. Go to the **GCP Console → Storage → Transfer**.
2. Click "**Create Transfer Job**".
3. Choose:
   - **Source**: Amazon S3
   - **Destination**: GCS Bucket
4. Provide:
   - AWS Access Key and Secret
   - S3 bucket name
   - GCS bucket name
5. Schedule transfer (One-time or recurring)
6. Review and create.

> GCS Transfer Service handles reliability, retries, and scheduling automatically.

✅ 3. Using `rclone` (Cross-cloud open-source CLI)

```bash
# Configure rclone for both AWS and GCS
rclone config

# Transfer data
rclone copy aws_s3:your-s3-bucket gcs:your-gcs-bucket --progress
```

✅ 4. Using Dataflow (for advanced ETL use cases)

- Use **Apache Beam with Dataflow** to read from S3 and write to GCS.
- Best suited if transformation is needed during migration.

✅ 5. Third-Party Tools

- **CloudSync**, **CloudM**, **CloudEndure**, or **Storage Gateway**
- Used in enterprise setups for seamless migration or hybrid cloud.

📌 Choose Method Based on Use Case:

Use Case	Recommended Tool
One-time transfer (small/medium)	`gsutil`, `rclone`
Large-scale scheduled transfers	GCS Transfer Service
ETL during transfer	Dataflow / Apache Beam
Hybrid cloud/GUI-based	Third-party tools

118.How to copy files from GCS bucket of one project to another project. If you do not have access and we are provided with a service account that has access?

To **copy files from a GCS bucket in one project to another project**, using a **service account (SA)** that has access to the source bucket, follow these steps:

✅ **Scenario**

- You do **not** have direct access to source bucket.
- You're provided a **service account key file (.json)** that has access.
- You want to **copy data from source bucket (Project A)** to your **destination bucket (Project B)**.

🔧 Steps

1. Activate the Service Account

```bash
gcloud auth activate-service-account --key-file=/path/to/service-account-key.json
```

This allows your local session to impersonate the service account.

---

2. (Optional) Set the Source Project

If needed (mostly for quota), you can set the source project temporarily:

```bash
gcloud config set project <SOURCE_PROJECT_ID>
```

---

3. Use `gsutil` to Copy Files

```bash
gsutil -m cp -r gs://<SOURCE_BUCKET_NAME>/<path> gs://<DESTINATION_BUCKET_NAME>/
```

- `-m`: Enables parallel (multi-threaded) copy.
- `-r`: Recursive copy if copying folders.

> Ensure the **destination bucket exists** in your project and you have **write access** to it.

✅ Example:

```bash
gcloud auth activate-service-account --key-file=source-sa.json

gsutil -m cp -r gs://source-bucket/data/ gs://destination-bucket/data/
```

🛡️ Permissions Required:

The **service account** must have these roles on the **source bucket**:

- `roles/storage.objectViewer` (to read files)
- `roles/storage.legacyBucketReader` (optional for listing bucket contents)

You (your identity) must have write access to the **destination bucket**.

119.What is the difference between gsutil and gcloud?

✅ When to Use What?

Task	Use gsutil	Use gcloud
Upload/download files to/from GCS	✅ Best suited	☑️ Basic support
Manage all GCP services (VMs, BQ, IAM)	❌	✅
Automate GCS object-level tasks in scripts	✅	☑️
Manage GCS buckets (create, delete, list)	✅	✅
IAM roles, auth, billing, networking	❌	✅

🔷 `gsutil`

- **Purpose:** Specifically for interacting with **Google Cloud Storage (GCS)**.
- **Tool Type:** Python-based command-line tool.
- **Common Use Cases:**
  - Upload/download files to/from GCS.
  - Set bucket and object permissions.
  - Move, copy, or delete GCS objects.
  - Configure lifecycle rules and storage classes.
- **Examples:**

```bash
gsutil cp file.txt gs://my-bucket/
gsutil mv gs://my-bucket/file.txt gs://my-bucket/archive/
gsutil rm gs://my-bucket/oldfile.txt
```

🔷 `gcloud`

- **Purpose:** A more **comprehensive CLI** tool to manage **all Google Cloud services** (including but not limited to GCS).
- **Tool Type:** Official CLI for GCP.
- **Common Use Cases:**
  - Manage GCS (limited file operations).
  - Deploy Cloud Functions, manage Compute Engine, BigQuery, IAM, etc.
  - Set project-level configurations, billing, auth.

Examples:

```bash
gcloud storage buckets create gs://my-bucket --location=us
gcloud compute instances list
gcloud auth login
```

119.How to check the hidden files Files of a bucket?

To check **hidden files** in a **Google Cloud Storage (GCS) bucket**, it's important to understand that:

> **GCS does not support hidden files** in the traditional sense (like files starting with a dot `.` in Unix-b systems).
> However, **files prefixed with** `.` (like `.env`, `.hidden_file`, etc.) **are treated as regular objects** in GC

✅ Ways to List Hidden Files in a GCS Bucket

1. Using `gsutil ls` with wildcard

bash Copy

```
gsutil ls gs://your-bucket-name/**/.*
```

- This lists files starting with `.` (dot) in any folder.
- It recursively searches for hidden files across all directories.

2. Using `gsutil ls` and `grep`

bash Copy

```
gsutil ls -r gs://your-bucket-name/ | grep '/\.[^/]*$'
```

- `-r`: Recursive listing
- `grep '/\.[^/]*$'`: Filters files that start with a dot (`.`) just before the filename.

120.How to create files in the GCS bucket using gsutil command?

To **create files in a GCS bucket** using the `gsutil` command, you typically follow a two-step process:

🛠️ Step 1: Create a local file (if not already available)

```bash
echo "Hello, GCS!" > testfile.txt
```

☁️ Step 2: Upload the file to a GCS bucket

```bash
gsutil cp testfile.txt gs://your-bucket-name/path/to/destination/
```

> ✅ Replace `your-bucket-name` with your actual bucket name and update the path as needed.

📝 Example

```bash
echo "Sample log data" > log.txt
gsutil cp log.txt gs://my-demo-bucket/logs/
```

121.What are the different ways to transfer data from GCS to Bigquery?

There are multiple ways to transfer data from Google Cloud Storage (GCS) to BigQuery, depending on the use case, data size, automation requirements, and integration with other tools.

✅ 1. Manual Load via BigQuery Console

- Steps:
  - Open BigQuery Console.
  - Click on the dataset > "Create Table".
  - Set source to GCS URI (gs://your-bucket/path).
  - Choose format (CSV, JSON, Parquet, Avro, ORC).
  - Define schema manually or auto-detect.
- Use case: One-time or ad hoc uploads.

✅ **2. bq Command-Line Tool**

- Example:

bash
CopyEdit
bq load \
 --source_format=CSV \
 my_dataset.my_table \
 gs://my-bucket/my-data.csv \
 ./schema.json

✅ **3. Using gcloud CLI (indirectly)**

- gcloud is more for GCS management, not loading directly to BigQuery, but can **trigger a workflow** that calls the bq CLI or API.

✅ **6. BigQuery Scheduled Queries with External Table (for Federated Load)**

- Use **external tables** for temporary access or transformations.
- Use CREATE TABLE AS SELECT to load from GCS-backed external table.
  sql
  CopyEdit
  CREATE OR REPLACE TABLE dataset.table AS
  SELECT * FROM EXTERNAL_QUERY('gs://bucket/path/*.parquet');

122.What are the different Storage classes available in GCS?

Google Cloud Storage (GCS) offers four different storage classes, each optimized for different access patterns and cost considerations:

1. **Standard Storage**
2. 2. **Nearline Storage**
3. Coldline Storage
4. **Archive Storage**

Storage Class	Access Frequency	Storage Cost	Retrieval Cost	Use Case
Standard	Frequent	High	None	Active data, websites
Nearline	Monthly	Medium	Moderate	Backups, disaster recovery
Coldline	Quarterly	Low	Higher	Long-term backup, archive
Archive	Yearly or less	Lowest	Highest	Legal archives, deep storage

🔹 16. Dataform - Real-Time Scenarios

Q123: How do you modularize your Dataform project for better maintainability?

Answer:

- Use separate .sqlx models for each transformation step.
- Create **dependencies** via ref():
  js
  CopyEdit
  config {
  type: "table"
  }
  SELECT * FROM ${ref('stg_customer')}
- Group related models into directories: /staging/, /marts/, /intermediate/.

124: You need to implement incremental models in Dataform. How would you do it?

Answer:
sql
CopyEdit
config {
type: "incremental",
uniqueKey: "id",

```
  incrementalQueryFilter: "updated_at > (SELECT MAX(updated_at) FROM ${self})"
}
SELECT * FROM source_table
```

125: How do you manage different environments (dev, prod) in Dataform?

Answer:

- Use **environments. Json** and **dataform.json** for configuration.
- Reference projectConfig.defaultSchema or environment.name in logic:
  sql
  CopyEdit
  SELECT * FROM ${ref("my_table", "prod")}

126: How would you load multi-format files (CSV, JSON, Avro) from GCS to BigQuery dynamically?

Answer:

- Use **Cloud Functions** triggered on GCS file upload.
- Inside the function, detect file extension and use bq load or Python client accordingly:
  python
  CopyEdit
  if file.endswith(".csv"):
  source_format = "CSV"
  elif file.endswith(".json"):
  source_format = "NEWLINE_DELIMITED_JSON"
  elif file.endswith(".avro"):
  source_format = "AVRO"

127: How would you perform a cost-effective full outer join of two huge tables in BigQuery?

Answer:

- Filter unnecessary columns early.
- Materialize intermediate steps if reused.
- Use partitioned and clustered tables.
- If possible, break into smaller JOINs (e.g., LEFT then RIGHT EXCEPT LEFT).

128: How would you validate your transformations before deploying to production in Dataform?

Answer:

- Use dataform test and run **unit tests**:
  js
  CopyEdit
  config {
  type: "assertion"
  }
  SELECT * FROM ${ref('stg_customers')} WHERE customer_id IS NULL
- Use GitHub Actions or Cloud Build for **CI/CD** with dataform run --dry-run.

129: How do you manage schema drift (e.g., new columns) in upstream data using Dataform?

Answer:

- Use SELECT * EXCEPT(unwanted_column) and avoid hardcoding column names.
- Monitor schema with information_schema.columns in validation queries.
- Create a generic fallback table and flag differences.

130: How would you ensure downstream tables are refreshed only if upstream tables change?

Answer:

- Use dependencies: explicitly in config and model DAG structure via ref().
- Use incremental logic to only query new/changed data.
- Implement **run conditions** in your workflow (e.g., hash comparisons).

Here are **real-time custom scheduling scenarios** in **Apache Airflow** using **schedule_interval** with cron expressions and timedelta options:

$$$

Cloud Composer And DAG Interview Questions and Answers

✅ **131. Scenario: Data Dependency Between Tasks**

Q: You have 3 tasks: A → B → C. Task A and B are fast, but C takes hours. You need to skip C when not required. How do you design this in Airflow?
A:
Use the **BranchPythonOperator** to conditionally skip Task C:

python

CopyEdit

```
def decide_next(**kwargs):
    if kwargs['dag_run']. conf.get('run_task_c') == 'yes':
        return 'task_c'
    else:
        return 'end'
branch = BranchPythonOperator(
    task_id='branch_decision',
    python_callable=decide_next,
    provide_context=True,
```

```
    dag=dag
)
task_c = PythonOperator (task_id='task_c', python_callable=run_c, dag=dag)
end = DummyOperator (task_id='end', dag=dag)
a >> b >> branch
branch >> [task_c, end]
```

✅ 132. Scenario: Rerun Failed Task Only

Q: You have a DAG with 10 tasks. One task fails. How do you rerun only that failed task without rerunning the entire DAG?
A:
Use the Airflow CLI or UI:

bash

CopyEdit

```
airflow tasks retry <dag_id> <task_id> --execution-date <date>
```

Or in the UI, click on the failed task and select **"Clear"** → **"Upstream: False, Downstream: False"** to rerun only that task.

✅ 133. Scenario: Task Failure Notification

Q: How do you notify stakeholders when a task fails?
A:
Use email_on_failure=True, or a failure callback:

python

CopyEdit

```
def notify_slack(context):
    # Send message to Slack or Email
    pass
task = PythonOperator(
    task_id='critical_task',
    python_callable=job,
    on_failure_callback=notify_slack,
    dag=dag
)
```

✅ 134. Scenario: Task Dependency from Another DAG

Q: Task in DAG-B should start only after DAG-A finishes. How can you enforce this?
A:
Use **ExternalTaskSensor** in DAG-B:

python

CopyEdit

```
ExternalTaskSensor(
   task_id='wait_for_dag_a',
   external_dag_id='dag_a',
   external_task_id='final_task_a',
   mode='poke',
   timeout=600,
   dag=dag_b
)
```

✅ **135. Scenario: Retry Policy for Transient Failures**

Q: Your task fails due to temporary DB connection issues. How do you set retry behavior?
A:
Use retries, retry_delay, and retry_exponential_backoff:

python

CopyEdit

```
PythonOperator(
   task_id='load_data',
   python_callable=load_fn,
   retries=3,
   retry_delay=timedelta(minutes=5),
   retry_exponential_backoff=True,
   dag=dag
)
```

✅ **136. Scenario: Parallel Task Execution**

Q: How can you run multiple tasks in parallel within a DAG?
A:
Just don’t chain them. Example:

python

CopyEdit

```
start = DummyOperator(task_id='start', dag=dag)

task1 = PythonOperator(task_id='task1', python_callable=run1, dag=dag)

task2 = PythonOperator(task_id='task2', python_callable=run2, dag=dag)


start >> [task1, task2]
```

Ensure max_active_runs and concurrency are set properly in the DAG.

✅ **137. Scenario: Skipping Weekends**

Q: How do you configure your DAG to not run on weekends?
A:
Use schedule_interval='0 9 * * 1-5' to schedule Mon-Fri 9 AM, or skip execution within the DAG:

```
python

CopyEdit

def skip_weekends(**kwargs):
    import datetime
    if datetime.datetime.today().weekday() >= 5:
        raise AirflowSkipException("It's weekend!")


check_day = PythonOperator(
    task_id='check_day',
    python_callable=skip_weekends,
    provide_context=True,
    dag=dag
)
```

✅ **138. Scenario: Data Integrity Between Upstream & Downstream**

Q: Your downstream task depends on all upstream tasks completing successfully. How do you enforce this?
A:
Airflow handles this by default, but ensure TriggerRule.ALL_SUCCESS is set:

```
python

CopyEdit

final_task = PythonOperator(
    task_id='final',
    python_callable=do_something,
```

```
    trigger_rule='all_success',
    dag=dag
)
```

✅ **139. Scenario: Trigger a DAG on File Upload**

Q: You want to trigger a DAG when a new file is uploaded to a GCS bucket. How will you implement this?
A:
Use **GCS Sensor** or **Event-based trigger** in Cloud Composer:

```
python
CopyEdit
GCSSensor(
    task_id='wait_for_file',
    bucket='my-bucket',
    object='data/{{ ds }}/input.csv',
    mode='poke',
    poke_interval=60,
    timeout=3600,
    dag=dag
)
```

Or use **Pub/Sub + Cloud Function** to trigger the DAG via REST API.

140.What are the Core Components of Airflow?

The core components of **Apache Airflow** are essential for managing, scheduling, and executing workflows (DAGs). Here's a breakdown of the **core components**:

🔹 **1. DAG (Directed Acyclic Graph)**

- A collection of all the tasks you want to run, organized in a way that reflects their relationships and dependencies.
- Written in Python code.
- Defines the **workflow** structure.

🔹 **2. Operators**

- Define a **single task** in a DAG.
- Examples:
    - PythonOperator – runs Python code.
    - BashOperator – runs bash commands.
    - DummyOperator – placeholder task.

- EmailOperator, SqlOperator, etc.

🔹 **3. Tasks**

- A **specific instance** of an Operator.
- Represents a unit of work to be executed.

✅ **141. Scenario: Data Dependency Between Tasks**

- **Q: You have 3 tasks: A → B → C. Task A and B are fast, but C takes hours. You need to skip C when not required. How do you design this in Airflow?**
  A:
  Use the BranchPythonOperator to conditionally skip Task C:
- python
- CopyEdit
- def decide_next(**kwargs):
- if kwargs['dag_run'].conf.get('run_task_c') == 'yes':
- return 'task_c'
- else:
- return 'end'
-
- branch = BranchPythonOperator(
- task_id='branch_decision',
- python_callable=decide_next,
- provide_context=True,
- dag=dag
-)
-
- task_c = PythonOperator(task_id='task_c', python_callable=run_c, dag=dag)
- end = DummyOperator(task_id='end', dag=dag)

🔹 **4. Scheduler**

- Responsible for:
  - Monitoring DAG definitions.
  - Scheduling the execution of tasks based on DAG schedule intervals.
  - Queues tasks for execution.

🔹 **5. Executor**

- Responsible for **running the tasks**.
- Types of Executors:
  - SequentialExecutor (for testing/single-threaded)
  - LocalExecutor
  - CeleryExecutor (for distributed execution)
  - KubernetesExecutor

🔹 **6. Worker**

- Executes the tasks.
- Relevant when using distributed executors like Celery or Kubernetes.

🔹 **7. Web Server**

- Provides the **Airflow UI**.
- Allows users to:
    - Monitor DAGs
    - Trigger DAGs manually
    - Check logs
    - Manage task states

🔹 **8. Metadata Database**

- Stores:
    - DAG definitions
    - Task states (success, failed, etc.)
    - Schedules and execution logs
- Backend: usually **PostgreSQL** or **MySQL**

🔹 **9. CLI (Command Line Interface)**

- Allows users to interact with Airflow.
- Examples:
    - airflow dags list
    - airflow tasks run
    - airflow db init

142.How Does Airlfow Works?

Apache **Airflow** is an open-source tool for orchestrating workflows — it helps you schedule, monitor, and manage data pipelines. Here's a **simple explanation of how Airflow works**:

🔁 **How Airflow Works (Step-by-Step)**

Step	Component	What Happens
1️⃣	**DAG Definition**	You write a **DAG (Directed Acyclic Graph)** using Python to define tasks and their order.
2️⃣	**Scheduler**	The **Scheduler** reads DAGs and schedules the task instances at the right time (based on schedule_interval).
3️⃣	**Metadata DB**	Airflow stores the status of each DAG run and task instance in a **Metadata Database** (like PostgreSQL/MySQL).
4️⃣	**Executor**	The **Executor** determines how and where to run each task (locally, in Celery workers, Kubernetes, etc.).
5️⃣	**Worker** (Optional)	**Workers** actually **execute the tasks** (e.g., running a Python function, Bash script, SQL query, etc.).
6️⃣	**Web UI**	You can monitor DAGs, view logs, and trigger or pause workflows using Airflow's **Web Interface**.

🧠 **Example:**

python

CopyEdit

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

with DAG('sample_dag', start_date=datetime(2023, 1, 1), schedule_interval='@daily') as dag:
  task1 = BashOperator(
    task_id='print_hello',
    bash_command='echo "Hello World!"'
  )
```

- Airflow reads this DAG.
- Scheduler triggers it daily.
- Executor sends it to a worker.
- The worker runs echo "Hello World!"
- Logs and status are saved in the Metadata DB.

✅ **Key Features:**

- **Modular**: Write DAGs as Python code.
- **Pluggable Executors**: Choose between LocalExecutor, CeleryExecutor, KubernetesExecutor, etc.
- **Monitoring**: Use Web UI or CLI to monitor task status.
- **Retry & Alerting**: Automatic retries, email alerts, SLA monitoring.

143.How to transfer data between task in the airflow?

In **Apache Airflow**, you can **transfer data between tasks** using **XComs (Cross-Communication)**. XComs allow tasks to share small amounts of data (e.g., strings, dictionaries, numbers). Here's how it works:

✅ **Step-by-Step: Transferring Data Between Tasks Using XCom**

1. Push data from one task (producer):

Use xcom_push() inside a PythonOperator task:

python

CopyEdit

```
from airflow.operators.python import PythonOperator


def push_data(**kwargs):
  kwargs['ti'].xcom_push(key='my_key', valuexz='Hello from task_1')
```

```
push_task = PythonOperator(
    task_id='push_task',
    python_callable=push_data,
    provide_context=True,
    dag=dag
)
```

---

2. Pull data in another task (consumer):

Use xcom_pull() in the downstream task:

python

CopyEdit

```
def pull_data(**kwargs):
    value = kwargs['ti'].xcom_pull(key='my_key', task_ids='push_task')
    print(f"Received value: {value}")

pull_task = PythonOperator(
    task_id='pull_task',
    python_callable=pull_data,
    provide_context=True,
    dag=dag
)
```

🧠 **Notes:**

- task_ids: Specifies from which task to pull the data.
- key: The identifier used to push and pull the data.
- XComs are stored in the Airflow metadata database.
- Best suited for **small data** (e.g., config, strings, IDs)—**avoid large files or datasets**.

✅ **Alternative: Using return from PythonOperator (Airflow 2.x+)**

Airflow 2.x+ lets you use return values from PythonOperator, which automatically pushes to XCom:

```
python
CopyEdit
def push_return():
    return "Hello via return"

def pull_return(**kwargs):
    value = kwargs['ti'].xcom_pull(task_ids='push_return_task')
    print(value)

push_return_task = PythonOperator(
    task_id='push_return_task',
    python_callable=push_return,
    dag=dag
)

pull_return_task = PythonOperator(
    task_id='pull_return_task',
    python_callable=pull_return,
    provide_context=True,
    dag=dag
)
```

144.What is ExternalTaskSensor?

In **Apache Airflow**, the **ExternalTaskSensor** is a special kind of **sensor** used to **wait for a task in a different DAG to complete** before continuing execution in the current DAG.

✅ **Purpose:**
To **coordinate dependencies across multiple DAGs**. For example, if **DAG B** should only run **after a specific task in DAG A** is successful, you can use ExternalTaskSensor in DAG B.

🧠 **Key Parameters:**

Parameter	Description
external_dag_id	The DAG ID of the **external DAG** to wait on.
external_task_id	The **task ID** in the external DAG to wait for.

Parameter	Description
allowed_states	List of states (like ['success']) in which the external task is considered complete.
execution_date_fn	Function to map the current DAG run's execution date to the external DAG run's date (used when scheduling doesn't align).
Timeout	Optional. How long to wait before failing.
Mode	'poke' (default) or 'reschedule'. 'reschedule' is more efficient for long waits.

🔧 **Example:**

Let's say we have:

- **DAG A** with a task generate_data
- **DAG B** should wait until generate_data in DAG A is complete

In **DAG B**:

python

CopyEdit

```
from airflow.sensors.external_task import ExternalTaskSensor
from airflow import DAG
from datetime import datetime

with DAG('dag_b', start_date=datetime(2023, 1, 1), schedule_interval='@daily') as dag:

  wait_for_dag_a = ExternalTaskSensor(
    task_id='wait_for_generate_data',
    external_dag_id='dag_a',
    external_task_id='generate_data',
    allowed_states=['success'],
    mode='reschedule',
    timeout=600,
  )
```

📌 **Use Cases:**

- Data dependency between different pipelines
- Modular DAG design (decoupling ETL stages)
- Managing dependencies in a microservices-style DAG architecture
-

145.How we can create DAG to DAG dependency?

In **Apache Airflow**, to create **DAG-to-DAG dependencies**, you typically use:

✅ **Option 1: ExternalTaskSensor (Upstream DAG → Downstream DAG)**

This is the most common method to **make one DAG wait for another DAG to complete**.

Example:

Let's say:

- dag_A runs first.
- dag_B should wait for dag_A to complete.
  In dag_B.py:
  python
  CopyEdit
  from airflow.sensors.external_task import ExternalTaskSensor
  from airflow import DAG
  from datetime import datetime
  from airflow.operators.dummy import Dummy Operator

  with DAG(
    dag_id="dag_B",
    start_date=datetime(2023, 1, 1),
    schedule_interval="@daily",
    catchup=False
) as dag:

    wait_for_dag_A = ExternalTaskSensor(
      task_id="wait_for_dag_A",
      external_dag_id="dag_A",
      external_task_id="final_task_in_dag_A", # Use the last task ID in dag_A
      mode='poke', # or 'reschedule'
      timeout=600,
      poke_interval=30,
      dag=dag
)

    run_dag_B = DummyOperator(task_id="run_dag_B")

    wait_for_dag_A >> run_dag_B

✅ **Option 2: Trigger Another DAG Programmatically using TriggerDagRunOperator**
You can use this to **trigger another DAG from within a task** in the current DAG.
In dag_A.py:
python
CopyEdit
from airflow.operators.trigger_dagrun import TriggerDagRunOperator

trigger_dag_B = TriggerDagRunOperator(
  task_id="trigger_dag_B",
  trigger_dag_id="dag_B", # Name of the DAG to trigger
)
💡 You can combine both methods: dag_A triggers dag_B, and dag_B still uses ExternalTaskSensor to confirm successful completion of a specific task in dag_A.

Summary Table:

Method	Use Case	Direction	Notes
ExternalTaskSensor	Wait for another DAG/task to finish	Downstream	Ensures proper sequencing
TriggerDagRunOperator	Trigger another DAG	Upstream	Starts another DAG programmatically

92.How to run two task parallel in airflow?

To run **two tasks in parallel in Apache Airflow**, you just need to ensure:

1. They **do not have a dependency on each other** (i.e., no >> or << relationship).
2. The DAG's configuration and Airflow's executor allow parallelism.

✅ **Example DAG with Two Parallel Tasks**

python
CopyEdit

```
from airflow import DAG
from airflow.operators.dummy import DummyOperator
from datetime import datetime

with DAG(
    dag_id='parallel_tasks_example',
    start_date=datetime(2024, 1, 1),
    schedule_interval=None,
    catchup=False
) as dag:

    start = DummyOperator(task_id='start')

    task1 = DummyOperator(task_id='task1')
    task2 = DummyOperator(task_id='task2')

    end = DummyOperator(task_id='end')

    # Define parallel tasks
    start >> [task1, task2] >> end
```

🧠 **Key Concept:**

- start >> [task1, task2]: This syntax sends both tasks from the start node **in parallel**.
- Both task1 and task2 will run at the same time if resources (workers, slots) are available.

🛠️ **Additional Requirements for Parallel Execution**

- Ensure your **executor supports parallelism** (e.g., LocalExecutor, CeleryExecutor, or KubernetesExecutor).
- Check the following Airflow configs:

- parallelism in airflow.cfg (max concurrent tasks across all DAGs)
- dag_concurrency (max tasks per DAG)
- max_active_tasks_per_dag
- max_active_runs_per_dag

146.Write a Dag to detect a file in bucket and trigger a spark job subsequently in airflow ?

Here is a sample Airflow DAG to **detect a file in a GCS bucket** and **trigger a Spark job** when the file is found.

🧠 **Overview:**

- **Sensor Task**: Waits for a specific file in GCS using GCSObjectExistenceSensor
- **Spark Task**: Triggers a Spark job via SparkSubmitOperator or DataprocSubmitJobOperator (depending on environment)

✅ **Requirements:**

- Airflow with google and spark provider packages installed
- GCP connection setup in Airflow (google_cloud_default)
- Spark setup (locally or Dataproc on GCP)

✨ **DAG: File Detection + Spark Trigger**

python
CopyEdit

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.providers.google.cloud.sensors.gcs import GCSObjectExistenceSensor
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator

default_args = {
    'owner': 'airflow',
    'start_date': datetime(2024, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5)
}

with DAG(
    dag_id='gcs_file_detect_trigger_spark',
    schedule_interval='@hourly',
    default_args=default_args,
    catchup=False,
    description='Detect file in GCS and trigger Spark job',
) as dag:

    # 1. Wait for file in GCS
    wait_for_file = GCSObjectExistenceSensor(
        task_id='wait_for_gcs_file',
        bucket='your-gcs-bucket-name',
        object='path/to/your/file.csv',
        google_cloud_conn_id='google_cloud_default',
        timeout=600,
```

```
    poke_interval=30,
    mode='poke'
)

# 2. Trigger Spark job (Assuming local Spark)
trigger_spark_job = SparkSubmitOperator(
    task_id='run_spark_job',
    application='/path/to/your_spark_script.py',
    conn_id='spark_default',
    verbose=True,
    application_args=["gs://your-gcs-bucket-name/path/to/your/file.csv"]
)

wait_for_file >> trigger_spark_job
```

147.Can we execute the python and shell scripting in the same DAG ?

Yes, **you can execute both Python and Shell scripting in the same Airflow DAG**.
How?
Airflow allows multiple types of operators in a single DAG. You can use:

- PythonOperator to run Python code.
- BashOperator to run shell commands.

✅ **Example DAG with both Python and Shell tasks:**

```
python
CopyEdit
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from datetime import datetime

# Define Python function
def print_hello():
    print("Hello from Python task!")

with DAG(
    dag_id="python_shell_mixed_dag",
    start_date=datetime(2023, 1, 1),
    schedule_interval="@daily",
    catchup=False
) as dag:

    python_task = PythonOperator(
        task_id="python_task",
        python_callable=print_hello
    )

    shell_task = BashOperator(
        task_id="shell_task",
```

```
    bash_command="echo 'Hello from Shell script!'"
)

# Define task order
python_task >> shell_task
```

🔍 **Key Points:**

- Both tasks are part of the same DAG.
- You can control their order using >> or set_downstream.
- You can also mix other operators like EmailOperator, SqlOperator, SparkSubmitOperator, etc.

148.What is connection string in cloud composer for connecting bigquery ?

In **Cloud Composer (Airflow on GCP)**, you connect to **BigQuery** using a **Connection** of type Google Cloud.
There is no traditional *connection string* like in JDBC or ODBC; instead, Airflow uses a **connection object** defined in its metadata DB. Here's how it's set up and used:

✅ **Steps to Create BigQuery Connection in Cloud Composer (Airflow UI)**

1. **Go to Airflow UI** → Admin → Connections.
2. Click **"+ Add a new record"**.
3. Fill in the details:
    - **Conn Id**: my_gcp_connection (You'll refer to this in your DAGs)
    - **Conn Type**: Google Cloud
    - **Project Id**: Your GCP project ID (e.g., my-gcp-project)
    - **Keyfile Path** or **Keyfile JSON**: *(Optional)* Provide the service account credentials if Composer doesn't use default GCP credentials.
    - **Scopes**: Leave blank (default scopes are used), or use https://www.googleapis.com/auth/bigquery

---

✅ **How to Use This Connection in a DAG**

Example using BigQueryInsertJobOperator:

python

CopyEdit

```
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

bq_task = BigQueryInsertJobOperator(
    task_id="run_bq_query",
    configuration={
        "query": {
            "query": "SELECT * FROM `my_dataset.my_table` LIMIT 10",
            "useLegacySql": False,
        }
    },
```

```
    gcp_conn_id="my_gcp_connection",  # <-- This is the Airflow connection ID
)
```

📝 **Notes:**

- If **Cloud Composer** environment has permission to access BigQuery (via its service account), you don't need to provide a key file.
- The connection uses **Application Default Credentials (ADC)** when no key is provided.

149.How to check the Existence of a GCS file through Airflow ?

To **check the existence of a file in GCS (Google Cloud Storage) through Airflow**, you can use the GCSObjectExistenceSensor or GCSObjectExistenceAsyncSensor.

✅ **Option 1: Using GCSObjectExistenceSensor**

This sensor checks if a file (object) exists in a specific bucket.

python

CopyEdit

```
from airflow.providers.google.cloud.sensors.gcs import GCSObjectExistenceSensor

check_file_exists = GCSObjectExistenceSensor (
    task_id="check_gcs_file",
    bucket="your-bucket-name",
    object="path/to/your/file.csv",  # Replace with your actual file path
    timeout=300,                # Total time before failing
    poke_interval=10,           # How often to check (in seconds)
)
```

✅ **Option 2: Using GCSObjectExistenceAsyncSensor (for async DAGs)**

python

CopyEdit

```
from airflow.providers.google.cloud.sensors.gcs import GCSObjectExistenceAsyncSensor

check_file_exists = GCSObjectExistenceAsyncSensor (
    task_id="check_gcs_file_async",
    bucket="your-bucket-name",
    object="path/to/your/file.csv",
    timeout=300,
    poke_interval=10,
)
```

✅ **Option 3: Custom PythonOperator (alternative way)**

python

CopyEdit

```
from airflow.operators.python import PythonOperator
from google.cloud import storage

def check_file(bucket_name, file_path):
    client = storage.Client()
```

```
    bucket = client.get_bucket(bucket_name)
    blob = bucket.blob(file_path)
    if not blob.exists():
        raise FileNotFoundError(f"{file_path} not found in bucket {bucket_name}")

check_gcs_file_task = PythonOperator(
    task_id='check_file_python',
    python_callable=check_file,
    op_args=['your-bucket-name', 'path/to/your/file.csv'],
)
```

✅ **Best Practice**

- Use **GCSObjectExistenceSensor** when working in sync DAGs.
- Use **AsyncSensor** for performance in async DAGs.
- Set appropriate timeout and poke_interval.

150. How to backfill the data using Airflow?

In Apache **Airflow**, **backfilling** refers to the process of running a DAG for past dates (execution dates) to "fill in" missed or historical data processing.

✅ **How to Backfill Data in Airflow**

You can backfill a DAG using the airflow dags backfill command from the command line:

bash

CopyEdit

airflow dags backfill -s <start_date> -e <end_date> <dag_id>

🔍 **Parameters Explained:**

- -s or --start-date: The start date for backfilling (execution date).
- -e or --end-date: The end date for backfilling (execution date).
- <dag_id>: Your DAG's ID as defined in the DAG file.

🧠 **Example:**

Suppose you want to backfill data from **June 1 to June 5** for the DAG daily_sales_pipeline:

bash

CopyEdit

airflow dags backfill -s 2025-06-01 -e 2025-06-05 daily_sales_pipeline

This runs the DAG for each date from June 1 to June 5 **based on execution_date**, not the current date.

🧩 **Tips for Successful Backfill:**

1. **Make sure catchup=True** in your DAG if using scheduled intervals:

python

CopyEdit

DAG(..., catchup=True)

2. Ensure **start_date is set correctly** in your DAG — Airflow won't backfill for dates before the start_date.
3. Disable or manage depends_on_past=True if you don't want the tasks to wait for previous runs to complete.
4. Consider using --ignore-first-depends-on-past or --ignore-dependencies flags if dependencies are causing issues.

📌 **GUI Option:**

- In the Airflow UI, you can also **select past dates** in the DAG's graph view or calendar and click **"Run"** to backfill manually.

151.How will you manage Secrets and Sensitive Information in Cloud Composer?

In **Cloud Composer (Airflow on GCP)**, managing **secrets and sensitive information** securely is critical. Here are **multiple approaches** to manage secrets in Cloud Composer:

✅ **1. Using Environment Variables**

- **Store secrets** as environment variables in Cloud Composer environment.
- Access in DAGs via:
  python
  CopyEdit
  import os
  secret = os.environ.get("MY_SECRET")
- **Pros:** Simple, easily configurable in Composer UI.
- **Cons:** Visible to all users with access to environment.

✅ **2. Secret Manager (Recommended & Secure)**

- **Use Google Secret Manager** to store sensitive values like passwords, API keys.
- Access them in your DAG:
  python
  CopyEdit
  from google.cloud import secretmanager

```
def get_secret(secret_id):
    client = secretmanager.SecretManagerServiceClient()
    name = f"projects/{project_id}/secrets/{secret_id}/versions/latest"
    response = client.access_secret_version(name=name)
    return response.payload.data.decode("UTF-8")
```

- You need to:
    - Grant appropriate **IAM permissions** (Secret Manager Secret Accessor) to the Composer's service account.

✅ **3. Airflow Variables (with caution)**

- You can store secrets in Airflow Variables via UI or CLI:

```
bash
CopyEdit
gcloud composer environments run ENV_NAME \
  --location LOCATION variables -- set my_secret 'value'
```

- In DAG:

```
python
CopyEdit
from airflow.models import Variable
secret = Variable.get("my_secret")
```

- **Cons:** Stored in plain text in the Airflow metadata DB.

✅ **4. Using GCP KMS (Key Management Service)**

- Encrypt secrets using **Cloud KMS**, store encrypted value in GCS or Variables.
- Decrypt at runtime in DAG using KMS API.
- More complex, but highly secure.

✅ **5. Using Connection Password Fields**

- Store secrets (API tokens, passwords) in **Airflow Connections**.
- Use in hooks/operators:

```
python
CopyEdit
from airflow.hooks.base import BaseHook
conn = BaseHook.get_connection("my_conn_id")
api_key = conn.password
```

✅ **Best Practices Summary:**

Method	Secure	Recommended
Secret Manager	✅	✅✅✅
Airflow Connections	✅	✅✅
Environment Variables	⚠️	✅
Airflow Variables	⚠️	❌
KMS + GCS/Var (Advanced)	✅✅	✅✅

152.How would you handle the passing of large data between tasks in Cloud Composer using XComs? What best practices would you follow to ensure that XCom data is handled efficiently and doesn't affect the performance of your workflows?

Passing large data between tasks using **XComs** in **Cloud Composer (Airflow)** is **not recommended**, as XCom is designed for **small metadata**, not large payloads. Here's how to handle large data effectively and the **best practices** to follow:

❌ **Why Not Use XComs for Large Data?**

- **XComs store data in Airflow's metadata database** (PostgreSQL/Cloud SQL).
- Large payloads (e.g., big JSON blobs, dataframes, files) can:
  - Slow down the metadata DB.
  - Cause serialization issues.
  - Lead to performance degradation and DAG slowness.
  - Hit size limits and throw errors.

✅ **Best Practice: Use External Storage for Large Data**

Instead of pushing large data to XCom, **store it externally** and pass references (like file paths or object URLs) via XCom.

✅ **Recommended Approach**

1. **Store data in GCS** (Google Cloud Storage) from one task:

```
python
CopyEdit
def write_to_gcs(**kwargs):
    from google.cloud import storage
    client = storage.Client()
    bucket = client.bucket('my-bucket')
    blob = bucket.blob('path/to/data.json')
    blob.upload_from_string(json.dumps(large_data))

    # Push only the GCS path to XCom
    kwargs['ti'].xcom_push(key='gcs_path', value='gs://my-bucket/path/to/data.json')
```

2. **Read data from GCS in downstream task**:

```
python
CopyEdit
def read_from_gcs(**kwargs):
    from google.cloud import storage
    import json

    gcs_path = kwargs['ti'].xcom_pull(task_ids='write_task', key='gcs_path')
    bucket_name, blob_name = gcs_path.replace('gs://', '').split('/', 1)

    client = storage.Client()
    bucket = client.bucket(bucket_name)
    blob = bucket.blob(blob_name)
```

```
data = json.loads(blob.download_as_text())
```

📌 **Best Practices Summary**

Best Practice	Description
❌ **Avoid large XComs**	Do not use XCom to store large data like datasets, dataframes, or binaries.
✅ **Use GCS or BigQuery**	For data sharing between tasks, store data in GCS, BigQuery, or Pub/Sub.
✅ **Push references to XCom**	Only store file paths, IDs, or small metadata in XComs.
✅ **Set do_xcom_push=False**	Prevent unwanted data from being pushed into XCom in operators.
✅ **Enable XCom backend only if needed**	If using custom XCom backends (like GCS-backed), ensure it's well-tested.
🧹 **Clean up large temp files**	Remove GCS/temp data when no longer needed to save storage.

✅ **Example DAG Flow**

pgsql
CopyEdit
[Extract Task] --> stores data in GCS
|
v
[XCom push: GCS URI]
|
v
[Transform Task] --> reads data from GCS

153.How would you configure concurrency limits in Cloud Composer? What Airflow settings would you adjust to control the maximum number of parallel tasks within a DAG or across your entire Cloud Composer environment?

To configure **concurrency limits** in **Cloud Composer (Airflow)**, you can adjust several Airflow-level and DAG-level settings to control the number of parallel tasks that can run. These settings help you manage resources efficiently and avoid overwhelming your Composer environment.

🔧 **1. Environment-Level (Global) Concurrency Settings**

These apply across all DAGs:

- **parallelism** (Airflow config):
  - Maximum number of task instances that can run **across all DAGs** at once.

- Default: 32
- Set it in airflow.cfg via Cloud Composer > Environment Configuration:

ini
CopyEdit
[core]
parallelism = 64

- **dag_concurrency**:
  - Maximum number of task instances that can run **per DAG** simultaneously (total across all DAG runs).
  - Default: 16

ini
CopyEdit
[core]
dag_concurrency = 20

- **max_active_runs_per_dag**:
  - Maximum number of active DAG runs per DAG (DAG-level runs).
  - Default: 16

ini
CopyEdit
[scheduler]
max_active_runs_per_dag = 3

🗒️ **2. DAG-Level Concurrency Settings**

Set in the DAG definition:

python
CopyEdit
```
from airflow import DAG

dag = DAG(
   dag_id='my_dag',
   default_args=default_args,
   schedule_interval='@daily',
   max_active_runs=2,       # Max DAG runs at a time
   concurrency=10           # Max task instances from this DAG
)
```

- max_active_runs: Controls how many DAG **runs** can be active at once.
- concurrency: Limits how many **tasks** from this DAG can run concurrently, regardless of DAG runs.

🧱 **3. Task-Level Pooling (Optional for Fine Control)**

- Use **Pools** to limit concurrency at task level across DAGs.
- Create a pool in the Airflow UI (Composer > Airflow > Pools), then:

python
CopyEdit
```
PythonOperator(
   task_id='my_task',
   python_callable=my_function,
   pool='my_custom_pool',  # Pool with limited slots
```

```
    dag=dag
)
```

✅ **Best Practices**

- Monitor resource usage using **Cloud Monitoring** and **Airflow metrics**.
- Adjust worker_concurrency (for CeleryExecutor) if you're using that executor.
- Avoid setting very high parallelism without sufficient resources (Composer nodes).
- Use **Task Queues** and **Pools** if you have critical or heavy tasks.

154.How would you set up an external event trigger to start a Cloud Composer DAG?

To set up an **external event trigger** to start a **Cloud Composer DAG (Airflow DAG)**, you can use **Google Cloud services** like **Pub/Sub**, **Cloud Functions**, or **Cloud Storage** along with Airflow's **TriggerDagRunOperator** or **REST API**. Here's a step-by-step approach using **Pub/Sub (most common method)**:

✅ **Use Case: Trigger a DAG in Cloud Composer when an external event happens (e.g., file uploaded to GCS, message sent to Pub/Sub)**

🔧 **Step-by-Step Setup (Using Pub/Sub + Cloud Function)**

Step 1: Create a Pub/Sub Topic

bash

CopyEdit

```
gcloud pubsub topics create trigger-dag-topic
```

Step 2: Create a Cloud Function

This function listens to the Pub/Sub topic and triggers your DAG via Airflow's REST API.

Python example:

python

CopyEdit

```
import base64
import json
import requests
from google.auth import default
from google.auth.transport.requests import Request

def trigger_dag(event, context):
    # Variables
    dag_id = "your_dag_id"
    project_id = "your_project_id"
    location = "your_composer_location"  # e.g., us-central1
    environment = "your_composer_environment_name"
    execution_date = None  # optional

    # Get credentials and Airflow endpoint
    credentials, _ = default(scopes=["https://www.googleapis.com/auth/cloud-platform"])
    credentials.refresh(Request())
```

```
    headers = {
        "Authorization": f"Bearer {credentials.token}",
        "Content-Type": "application/json"
    }

    url =
f"https://composer.googleapis.com/v1/projects/{project_id}/locations/{location}/environme
nts/{environment}/dagRuns"

    data = {
        "dagId": dag_id,
        "conf": {"message": base64.b64decode(event['data']).decode("utf-8")}
    }

    response = requests.post(url, headers=headers, json=data)
    print(response.status_code, response.text)
```

- Deploy this Cloud Function and link it to your Pub/Sub topic:

```
bash
CopyEdit
gcloud functions deploy triggerDagFromPubSub \
  --runtime python311 \
  --trigger-topic trigger-dag-topic \
  --entry-point trigger_dag \
  --region YOUR_REGION
```

Step 3: Configure your DAG to accept conf

In your Airflow DAG, enable configuration support:

```
python
CopyEdit
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def my_task(**kwargs):
    conf = kwargs['dag_run'].conf
    print("Received configuration:", conf)

with DAG(
    dag_id="your_dag_id",
    start_date=datetime(2023, 1, 1),
    schedule_interval=None,
    catchup=False,
) as dag:

    task = PythonOperator(
        task_id="print_conf",
        python_callable=my_task,
        provide_context=True,
```

)

✅ **Alternative Methods:**

- **Cloud Storage Event → Cloud Function → Trigger DAG**
- **Manual Trigger via Airflow REST API**
- **Cloud Scheduler → Trigger DAG on schedule with specific params**
- **Workflows or External System → Pub/Sub → Cloud Function → Trigger DAG**

🔐 **Security Tips:**

- Use **IAM service accounts** with appropriate Composer permissions.
- Restrict **Cloud Function triggers** to trusted sources.
- Validate the event data before using it in your DAG.

155.How to run multiple task parallel in Airflow?

To run **multiple tasks in parallel** in Apache Airflow, you can configure **DAG structure** and **Airflow settings** properly. Here's how:

✅ **1. Define Tasks Without Dependencies**

Tasks run in parallel **only if there are no upstream/downstream dependencies** between them.

Example:

python

CopyEdit

```
from airflow import DAG
from airflow.operators.dummy import DummyOperator
from datetime import datetime

with DAG('parallel_tasks_example', start_date=datetime(2023, 1, 1),
schedule_interval=None, catchup=False) as dag:

    start = DummyOperator(task_id='start')

    task1 = DummyOperator(task_id='task1')
    task2 = DummyOperator(task_id='task2')
    task3 = DummyOperator(task_id='task3')

    end = DummyOperator(task_id='end')

    start >> [task1, task2, task3] >> end  # These run in parallel
```

✅ **2. Set DAG-Level Concurrency Settings**

In the DAG definition:

python

CopyEdit

```
DAG(
    'parallel_tasks_example',
```

```
    default_args=default_args,
    schedule_interval=None,
    max_active_tasks=5,     # Max parallel tasks per DAG
    concurrency=5,          # Max active tasks in this DAG run
)
```

✅ **3. Set Global Parallelism in airflow.cfg**

Adjust these Airflow settings to allow more parallelism:

ini

CopyEdit

```
# In airflow.cfg
parallelism = 32        # Total parallel tasks across all DAGs
dag_concurrency = 16    # Tasks per DAG that can run concurrently
max_active_runs_per_dag = 1   # Number of DAG runs that can run in parallel
```

Or use Airflow environment variables if using Composer or KubernetesExecutor.

✅ **4. Executor Must Support Parallelism**

Make sure you're using a parallel-capable executor:

- **LocalExecutor** (limited parallelism)
- **CeleryExecutor**, **KubernetesExecutor**, or **DaskExecutor** for better scalability
- **SequentialExecutor** (default) does **not** allow parallelism

---

✅ **5. Optional: Use pool or priority_weight**

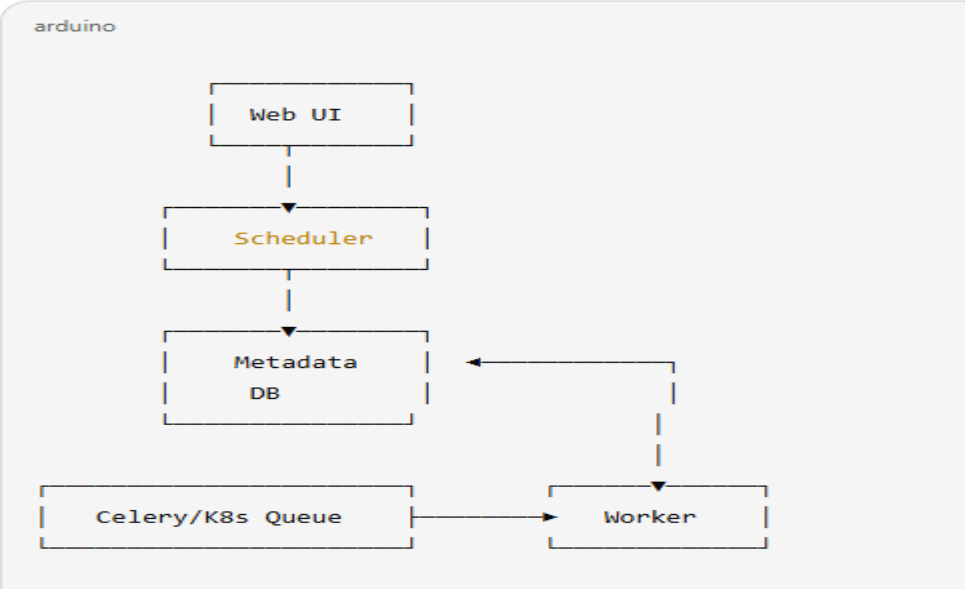
Control which tasks should get resources first when limits are hit.

Summary

Setting	**Purpose**
start >> [t1, t2, t3]	Allows tasks to run in parallel
max_active_tasks, concurrency	Controls parallelism within DAG
parallelism	Total parallel tasks in Airflow
Use CeleryExecutor	For scalable distributed execution

- **what is Architecture of Airflow?**

🧱 Airflow Architecture Diagram Overview:



📝 **Key Takeaways:**

- **Scheduler** triggers task instances.
- **Executor** determines how/where tasks run.
- **Web Server** is the monitoring and control interface.
- **Metadata DB** keeps everything persistent and consistent.
- **Workers** handle the actual execution of your task logic.

156.What are the parameters of PythonOperators and BashOpearators?

In Apache Airflow, PythonOperator and BashOperator are two commonly used operators. Below are the main parameters (arguments) for each, with short explanations:

Example :-

```
def print_hello(name):
    print(f"Hello, {name}")

task = PythonOperator(
    task_id='say_hello',
    python_callable=print_hello,
    op_kwargs={'name': 'Airflow'},
    dag=dag
)
```

✅ `PythonOperator` Parameters

python Copy Edit

```python
from airflow.operators.python import PythonOperator
```

Parameter	Description
`task_id`	Unique ID for the task in the DAG.
`python_callable`	The Python function you want to execute.
`op_args`	List of positional arguments to pass to the callable.
`op_kwargs`	Dict of keyword arguments to pass to the callable.
`provide_context`	*(Deprecated in Airflow 2)* Automatically passed context if True. Use `**kwargs` instead.
`retries`	Number of retries in case of task failure.
`retry_delay`	Time delay between retries.
`depends_on_past`	Task will run only if the previous instance succeeded.
`execution_timeout`	Max time allowed for task execution.
`dag`	The DAG this task is part of.
`queue`	Name of the queue to use if using CeleryExecutor.
`pool`	Resource pool to use.
`priority_weight`	Priority of task when using pools.
`trigger_rule`	How the task should be triggered (e.g., all_success).

✅ BashOperator Parameters
from airflow.operators.bash import BashOperator

Parameter	Description
`task_id`	Unique ID for the task.
`bash_command`	The bash command or script to execute.
`env`	Dict of environment variables to set.
`cwd`	Working directory to execute the command in.
`output_encoding`	Encoding to decode output (default is `utf-8`).
`append_env`	Whether to append to the existing environment.
`retries`	Number of retries if the task fails.
`retry_delay`	Time between retries.
`dag`	The DAG this task is part of.
`execution_timeout`	Maximum run time allowed for this task.
`depends_on_past`	Only run if the previous run succeeded.

```
task = BashOperator(
   task_id='print_date',
   bash_command='date',
   dag=dag
)
```

157.How would you configure Cloud Composer to automatically retry failed tasks?

To configure **Cloud Composer (Apache Airflow)** to automatically retry failed tasks, you can set retry parameters in the **task-level configuration** using the PythonOperator, BashOperator, or any other operator.

✅ **Key Parameters to Set for Retries:**

Parameter	Description
retries	Number of times to retry the task if it fails.
retry_delay	Delay between retries (as datetime.timedelta).
max_retry_delay	Maximum delay between retries (optional).
retry_exponential_backoff	If set to True, applies exponential backoff to retry delay.

✅ **Example: Configure Retries in a Task**

python

CopyEdit

```
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from datetime import datetime, timedelta
```

```
default_args = {
    'owner': 'airflow',
    'start_date': datetime(2025, 6, 1),
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
    'retry_exponential_backoff': True,
    'max_retry_delay': timedelta(minutes=60)
}

with DAG(
    dag_id='retry_example_dag',
    default_args=default_args,
    schedule_interval='@daily',
    catchup=False
) as dag:

    def my_function():
        raise Exception("Simulated task failure")

    task_with_retries = PythonOperator(
        task_id='task_retry_demo',
        python_callable=my_function
    )
```

🔍 **Notes:**

- If you define retries in both default_args and task-level, the task-level value takes precedence.
- You can monitor retries in the **Airflow UI**, where task instances will show up as **retrying** if they fail and are eligible for retry.

✅ **Real-World Tip (Cloud Composer):**

- Retry settings are handled by the Airflow scheduler and don't need additional GCP-specific configuration in Composer.
- You can also set task-level alerts for failure via email or GCP Pub/Sub if required.

158.What is Jinja Template and Dynamic dags in Airflow?

In Apache Airflow, Jinja templates and Dynamic DAGs are two powerful concepts that allow you to make your workflows flexible, reusable, and scalable.

🔶 What is Jinja Template in Airflow?

Jinja is a templating engine for Python. In Airflow, Jinja templating is used to dynamically insert variables and expressions into strings like bash_command, python_callable, or SQL queries.

✅ Use Case:

If you want to inject runtime values like execution date, task instance data, or custom parameters.

Example :-

```
from airflow.operators.bash import BashOperator

bash_task = BashOperator(
   task_id='print_execution_date',
   bash_command='echo {{ ds }}',
   dag=dag,
)
```

🔹 **Common Jinja Variables:**

- {{ ds }} – execution date (YYYY-MM-DD)
- {{ ds_nodash }} – execution date without dashes
- {{ macros.ds_add(ds, 1) }} – adds 1 day to ds
- {{ task_instance }} – task instance object
- {{ params.my_param }} – user-defined parameter

159. What are Dynamic DAGs in Airflow?

Dynamic DAGs refer to the concept of generating tasks or even entire DAGs programmatically using Python logic (like loops, conditions, config files, etc.).

✅ **Why Use It?**

When you have:

- Many similar tasks (e.g., processing 100 files or partitions)
- A config-driven pipeline
- Different DAGs for different clients or environments

✅ **Example (Dynamic Tasks):**

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

dag = DAG('dynamic_task_dag', start_date=datetime(2023, 1, 1), schedule_interval='@daily')

file_list = ['file1.csv', 'file2.csv', 'file3.csv']

for file in file_list:
   BashOperator(
      task_id=f'process_{file.replace(".csv", "")}',
      bash_command=f'echo "Processing {file}"',
      dag=dag
   )
```

✅ Example (Dynamic DAGs):

```
def generate_dag(client_id):
```

```
with DAG(
    dag_id=f'client_{client_id}_dag',
    schedule_interval='@daily',
    start_date=datetime(2023, 1, 1)
) as dag:
    BashOperator(
        task_id='print_client',
        bash_command=f'echo "Running DAG for client {client_id}"'
    )
return dag

for client in ['A', 'B', 'C']:
    globals()[f'client_{client}_dag'] = generate_dag(client)
```

Summary

Feature	Jinja Templates	Dynamic DAGs
Purpose	Insert dynamic content into strings	Generate tasks/DAGs programmatically
Syntax	`{{ variable }}`	Use Python logic (loops, functions)
Example Use	SQL queries, Bash commands, file names	Client-specific DAGs, file-based tasks
Benefits	Reusability, flexibility	Scalability, config-driven workflows

160: A task in your DAG fails intermittently due to API timeout. How do you handle this?
Answer:

- Use retry parameters in the operator:
  python
  CopyEdit
  PythonOperator(
  task_id='api_call',
  python_callable=my_api_func,
  retries=3,
  retry_delay=timedelta(minutes=5)
)
- Enable alerting with **EmailOperator** or Slack.

161: How do you backfill 1 month of data using a DAG?
Answer:

- Use catchup=True in DAG definition.
  python
  CopyEdit
  DAG(
  dag_id="backfill_dag",
  schedule_interval='@daily',

```
    start_date=datetime(2023, 5, 1),
    catchup=True
)
```

- Run:
  bash
  CopyEdit
  airflow dags backfill -s 2023-05-01 -e 2023-05-31 backfill_dag

---

162: How do you pass a large DataFrame between two tasks in Airflow?
Answer:

- Avoid XComs for large data.
- Instead, save to **GCS/BigQuery** in task 1 and reference path/table in task 2.
  python
  CopyEdit
  task1 = PythonOperator(...)
  task2 = PythonOperator(op_kwargs={'path': 'gs://bucket/df.csv'})

110: How do you create dependencies between two separate DAGs in Composer?
Answer:
Use ExternalTaskSensor in DAG-B:
python
CopyEdit

```
ExternalTaskSensor(
    task_id='wait_for_dag_a',
    external_dag_id='dag_a',
    external_task_id='final_step',
    mode='poke',
    poke_interval=60,
    timeout=600
)
```

163: How do you handle credential rotation (e.g., expiring keys) in Cloud Composer DAGs?
Answer:

- Store credentials (e.g., service account key) in **Secret Manager**.
- Retrieve them dynamically using:
  python
  CopyEdit

```
from google.cloud import secretmanager

def get_secret(secret_id):
    client = secretmanager.SecretManagerServiceClient()
    response = client.access_secret_version(name=f"{secret_id}/versions/latest")
    return response.payload.data.decode("UTF-8")
```

164: How do you execute a Spark job on Dataproc from Cloud Composer?
Answer:
Use DataprocSubmitJobOperator:

```
python
CopyEdit
DataprocSubmitJobOperator(
   task_id="spark_task",
   job=spark_job_config,
   region="us-central1",
   project_id="my-project"
)
```

GCP Cloud Composer / DAG Real Time Scenario Questions and Answers

Here are **Real-Time Scenario-Based Questions and Answers** for **GCP Cloud Composer** (Apache Airflow) and **DAGs** that are commonly encountered in GCP Data Engineer interviews and real-world projects.

In **Apache Airflow**, dependencies between tasks are handled using a **Directed Acyclic Graph (DAG)** structure. Here's a detailed step-by-step explanation of how Airflow manages dependencies:

✅ **165. Scenario: Schedule DAG to run on business days only**
Q: How do you schedule a DAG to run only on weekdays (Mon–Fri)?
Answer:
Use a cron expression that excludes weekends:
python
CopyEdit
schedule_interval = '0 6 * * 1-5' # Every weekday at 6 AM
You can also use pendulum to verify if execution date is a weekday before processing.

✅ **166. Scenario: Skip downstream if no new files found**
Q: If no new files are available in GCS, you want to skip processing tasks. How would you do this?
Answer:
Use a BranchPythonOperator to conditionally skip processing:
python
CopyEdit
def check_for_new_files():
   # logic to check GCS via GCS Hook
   return 'process_data' if new_files else 'skip_task'

BranchPythonOperator(
   task_id='check_files',
   python_callable=check_for_new_files
)

✅ **167. Scenario: DAG failure alert**
Q: How would you send an alert when a DAG or task fails?
Answer:

- Use email_on_failure=True in default args:
  python
  CopyEdit
  default_args = {
  'email': ['your-email@example.com'],
  'email_on_failure': True
  }
- Or use a **Slack webhook** via on_failure_callback.

✅ **168. Run DAG Every Day at 6:00 AM**
Schedule_interval = '0 6 * * *'
📌 Cron Explanation: minute hour day month day_of_week → 0 6 = 6:00 AM daily.

✅ **169. Run DAG Every Monday at 8:30 AM**
schedule_interval = '30 8 * * 1'
📌 Runs once a week every Monday at 8:30 AM.

✅ **170. Run DAG Every 15 Minutes**
schedule_interval = '*/15 * * * *'
📌 Triggers DAG every 15 minutes.

✅ **171. Run DAG Every 2 Hours**
schedule_interval = '0 */2 * * *'
📌 At the start of every 2nd hour: 00:00, 02:00, 04:00...

✅ **172. Run DAG on the First Day of Every Month at Midnight**
schedule_interval = '0 0 1 * *'
📌 12:00 AM on the 1st of each month.

✅ **173. Run DAG on Weekdays (Mon–Fri) at 7 PM**
schedule_interval = '0 19 * * 1-5'
📌 Monday to Friday at 7 PM.

✅ **174. Run DAG Every 45 Seconds (only possible with timedelta)**
from datetime import timedelta

schedule_interval = timedelta(seconds=45)
⚠️ Only works when catchup = False (recommended for short intervals).

✅ **175. Run DAG on the Last Day of Each Month at 11:55 PM**
Use a **cron preset** via Python logic because cron has no “last day” keyword directly.

from airflow.utils.dates import days_ago

```
def is_last_day(execution_date):
    next_day = execution_date + timedelta(days=1)
    return next_day.day == 1

# In DAG:
schedule_interval = '55 23 28-31 * *'  # Rough filter
```
⚠️ Then, in your task, check if it's the last day before proceeding.

✅ **176. Run DAG on the 15th and Last Day of Each Month at 8:00 PM**

```
schedule_interval = '0 20 15,L * *'
```
📌 Runs on the 15th and last day. (L is allowed in some cron interpreters like Quartz, but not standard Airflow cron. So use logic inside task if needed.)

✅ **177. Trigger Only Once**

```
schedule_interval = None
```
📌 This disables scheduling. Use manual or external triggers.

✅ **178. Run Every Day Except Sunday**

```
schedule_interval = '0 5 * * 1-6'
```
📌 Runs at 5 AM from Monday to Saturday.

✅ **179. Run Every 6 Hours Starting From Midnight**

```
schedule_interval = '0 */6 * * *'
```

✅ **180. Run DAG at Specific Times (e.g., 6AM, 12PM, 6PM)**

```
schedule_interval = '0 6,12,18 * * *'
```

Would you like me to create a table or a PDF summarizing these for your interviews or project documentation?

You said:

also please explain cleary step by step

ChatGPT said:

Sure! Here's a **clear, step-by-step explanation** of **how to schedule a DAG in Airflow using custom intervals**, with **real-time examples** and how to implement each step.

✅ **Step-by-Step Guide to Schedule a DAG in Airflow**

Step 1: Understand the schedule_interval Parameter

In your DAG definition, there's a parameter called schedule_interval which controls how often your DAG runs.

You can set:

- A **cron expression** (e.g., '0 6 * * *')
- A **timedelta object** (e.g., timedelta(hours=1))
- **None** for manual runs only

Step 2: Choose a Schedule Type

There are 3 common ways:

Type	Format Example	Meaning
cron string	'0 6 * * *'	Every day at 6 AM
timedelta	timedelta(hours=1)	Every hour
None	None	Run manually only

Step 3: Apply the schedule_interval in Your DAG
Here's the basic DAG structure:
python
CopyEdit

```
from airflow import DAG
from airflow.operators.dummy import DummyOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'airflow',
    'start date': datetime(2024, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5)
}

dag = DAG(
    'example_custom_schedule_dag',
    default_args=default_args,
    description='A DAG with custom schedule',
    schedule_interval='0 6 * * *',  # 👈 Replace with your schedule
    catchup=False
)

start = DummyOperator(
    task_id='start',
    dag=dag
)
```

✅ **Real-Time Scenarios with Explanation**

🔹 **Scenario 181: Run Every Day at 6 AM**
schedule_interval = '0 6 * * *'
Explanation:

- 0 = minute
- 6 = hour
- * * * = every day, every month, every weekday
- Runs daily at 6:00 AM

🔹 **Scenario 182: Run Every Monday at 8:30 AM**

schedule_interval = '30 8 * * 1'

Explanation:

- 30 = minute
- 8 = hour
- 1 = Monday (0 = Sunday, 6 = Saturday)

🔹 **Scenario 183: Run Every 15 Minutes**

schedule_interval = '*/15 * * * *'

Explanation:

- */15 = every 15 minutes
- Runs at :00, :15, :30, :45 of every hour

🔹 **Scenario 184: Run Every 2 Hours**

schedule_interval = '0 */2 * * *'

Explanation:

- */2 = every 2nd hour (0, 2, 4, ...)

🔹 **Scenario 185: Run on the First Day of Every Month at Midnight**

schedule_interval = '0 0 1 * *'

Explanation:

- 1 = 1st day of the month
- Runs at 12:00 AM

🔹 **Scenario 186: Run Every Weekday at 7 PM**

schedule_interval = '0 19 * * 1-5'

Explanation:

- 1-5 = Monday to Friday
- 19 = 7 PM (24-hour format)

🔹 **Scenario 187: Run Every 45 Seconds (Rare Case)**

from datetime import timedelta

schedule_interval = timedelta(seconds=45)

Note: Only supported when catchup=False. Not suitable for production unless needed.

🔹 **Scenario 188: Run on 15th and Last Day of the Month**

schedule_interval = '0 20 15,28-31 * *'

Use logic inside the task to check if it's the last day:

```
def is_last_day(execution_date):
    return (execution_date + timedelta(days=1)).day == 1
```

🔹 **Scenario 189: Run Every 6 Hours**

schedule_interval = '0 */6 * * *'

🔹 **Scenario 190: Run Only Once**

schedule_interval = None

Useful for one-time jobs or externally triggered DAGs.

✅ **Step 4: Disable Backfill if You Want Only Future Runs**

Set this in your DAG:
catchup=False
This avoids running past missed schedules.

✅ **Step 5: Deploy the DAG**

1. Save the .py DAG file in ~/airflow/dags/ or Cloud Composer's dags/ bucket.
2. Airflow will automatically detect the file and schedule the DAG based on your schedule_interval.

✅ **Summary Table of Cron Examples**

Expression	Description
'0 6 * * *'	Every day at 6 AM
'30 8 * * 1'	Every Monday at 8:30 AM
'*/15 * * * *'	Every 15 minutes
'0 */2 * * *'	Every 2 hours
'0 0 1 * *'	First day of month at midnight
'0 19 * * 1-5'	Weekdays at 7 PM
'0 20 15,28-31 * *'	15th and end of month at 8 PM
None	Manual trigger only

✅ **191. Task Dependencies via set_upstream() and set_downstream() or >> and << Operators**

You define the order of task execution using these methods:
Example:

```
task1 >> task2  # task2 runs after task1
task2 << task3  # task3 must complete before task2
```

This builds the task dependencies in the DAG graph.

✅ **192. DAG Structure: Directed Acyclic Graph**

- **Directed**: Dependencies have a direction — Task A -> Task B means A must run before B.
- **Acyclic**: Cycles are not allowed — a task cannot depend on itself either directly or indirectly.

✅ **193. Airflow Scheduler**

- The **Airflow Scheduler** evaluates which tasks are ready to run.
- A task will only be scheduled if **all its upstream dependencies are successful**.

✅ **194. Trigger Rules**

By default, tasks run when **all upstream tasks succeed**:

- Default rule: trigger_rule='all_success'

You can modify this:

Trigger Rule	Description
all_success	Run only if all upstream tasks succeeded
all_failed	Run only if all upstream tasks failed
one_success	Run if at least one upstream task succeeded
one_failed	Run if at least one upstream task failed
none_failed	Run if no upstream task failed
none_skipped	Run if no upstream task was skipped
Always	Run regardless of upstream task status

```
task3 = PythonOperator(
   task_id='task3',
   python_callable=my_func,
   trigger_rule='all_failed',  # will run only if upstream tasks failed
   dag=dag
)
```

✅ **195. Branching Dependencies**
Use BranchPythonOperator to direct the flow based on condition:

```
from airflow.operators.python import BranchPythonOperator

def choose_branch():
   return 'task_a' if condition else 'task_b'

branch = BranchPythonOperator(
   task_id='branching',
   python_callable=choose_branch,
   dag=dag
)

branch >> [task_a, task_b]
```

✅ **196. Cross-DAG Dependencies (ExternalTaskSensor)**
If task in **DAG-B** depends on completion of task in **DAG-A**:

```
from airflow.sensors.external_task import ExternalTaskSensor

wait_for_task = ExternalTaskSensor(
   task_id='wait_for_dag_a_task',
   external_dag_id='dag_a',
   external_task_id='task_a',
```

```
    dag=dag_b
)
```

✅ **197. Manual Dependency Management Using depends_on_past**

You can enforce that a task should only run if the previous run succeeded:

```
task1 = PythonOperator(
    task_id='task1',
    python_callable=my_func,
    depends_on_past=True,
    dag=dag
)
```

🔁 **Summary:**

Mechanism	Purpose
>>, <<, .set_upstream()	Define order of task execution
trigger_rule	Customize when a task should be triggered
depends_on_past	Enforce sequential task runs based on past
BranchPythonOperator	Conditional branching of task execution
ExternalTaskSensor	Cross-DAG dependencies

198.how, do you trigger DAG manually?

To **trigger a DAG manually in Airflow**, you can use any of the following methods:

✅ **1. Using Airflow Web UI**

Steps:

1. Go to the **Airflow Web UI** (typically at http://localhost:8080 or your Composer UI if using GCP).
2. In the **DAGs list**, find your DAG.
3. Click the **"Play" ▶️ button** next to the DAG name.
4. Choose a **start date** (optional) and click **"Trigger"**.

✅ **2. Using CLI**

Run this command from the terminal on the Airflow server:

```
airflow dags trigger <dag_id>
Example:
bash
CopyEdit
airflow dags trigger my_data_pipeline
```

To trigger it with a specific execution date:
bash
CopyEdit
airflow dags trigger -e 2024-01-01T00:00:00 my_data_pipeline

✅ **3. Using Python Script / REST API**
If you are using **Airflow 2.x**, you can use the **REST API**:

```
curl -X POST "http://<airflow-host>/api/v1/dags/<dag_id>/dagRuns" \
    -H  "Content-Type: application/json" \
    --user "airflow:your_password" \
    -d '{"conf": {}, "dag_run_id": "manual__trigger"}'
```

Or via Python (useful inside another DAG or automation script):

```
from airflow.api.client.local_client import Client

client = Client()
client.trigger_dag(dag_id='my_data_pipeline')
```

✅ **199. Scenario: Backfill Missed Data for a Specific Date Range**

Q: You deployed a new DAG that should have run for the last 7 days. How do you trigger a backfill for those dates?

A:
Use the Airflow CLI or UI to trigger a backfill:

bash

CopyEdit

airflow dags backfill -s 2025-06-01 -e 2025-06-07 my_dag_id

Or, enable catchup=True in the DAG definition:

python

CopyEdit

```
dag = DAG(
    dag_id="daily_data_pipeline",
    schedule_interval="@daily",
    start_date=datetime(2025, 6, 1),
    catchup=True
)
```

This ensures missed DAG runs are caught up for the range.

✅ 200. Scenario: Dynamically Generate DAGs for Multiple Clients

Q: You want to create DAGs for 10 different clients with similar pipelines. How would you implement it?

A:
Use Dynamic DAG generation using a for loop in a DAG factory file:

python

CopyEdit

```
for client in ['client_a', 'client_b', ..., 'client_j']:
    dag_id = f"process_data_{client}"
    default_args = {...}
    dag = DAG(dag_id, default_args=default_args, ...)

    with dag:
        task1 = BashOperator(task_id='extract', bash_command='echo extract')
        task2 = BashOperator(task_id='load', bash_command='echo load')
        task1 >> task2

    globals()[dag_id] = dag
```

✅ 201. Scenario: Retry Task on Failure with Alert

Q: A task might fail due to transient errors. You want to retry it and alert the team on final failure. How do you do this?

A:
python
CopyEdit

```
def on_failure_callback(context):
    send_email(to='devops@company.com', subject='Task Failed', html_content='See Airflow UI')

task = PythonOperator(
    task_id='unstable_task',
    python_callable=some_function,
    retries=3,
    retry_delay=timedelta(minutes=5),
    on_failure_callback=on_failure_callback,
    dag=dag
```

```
)
```

✅ 202. Scenario: Conditional Task Execution Based on Data Availability

Q: Run a task only if a file exists in GCS. How would you handle it?
A:
Use BranchPythonOperator to check file existence:

```
python
CopyEdit
def check_file(**kwargs):
    from google.cloud import storage
    client = storage.Client()
    bucket = client.get_bucket('my-bucket')
    blob = bucket.blob('path/to/file.csv')
    return 'process_data' if blob.exists() else 'skip_task'

branch = BranchPythonOperator(
    task_id='check_file',
    python_callable=check_file,
    provide_context=True,
    dag=dag
)
```

✅ 203. Scenario: Trigger DAG on File Upload in GCS

Q: How do you trigger a DAG when a file is uploaded to a GCS bucket?
A:

- Use **Cloud Function** + **Pub/Sub** to trigger DAG:
    1. Create Pub/Sub notification on the GCS bucket.
    2. Cloud Function receives event and calls Composer's REST API.
    3. Cloud Function example:

```
python
CopyEdit
import requests
def trigger_dag(request):
    requests.post(
        'https://composer-env-url/api/v1/dags/my_dag_id/dagRuns',
        json={"conf": {"filename": request['name']}},
        headers={"Authorization": "Bearer <token>"}
    )
```

✅ 204. Scenario: Parameterize DAG Using Airflow Variables

Q: How do you make your DAG accept project-specific or environment-specific parameters?
A:
Use Airflow Variables:

python
CopyEdit
from airflow.models import Variable

project_id = Variable.get("gcp_project_id")
bucket = Variable.get("data_bucket")

✅ 205. Scenario: Share Data Between Tasks

Q: How do you pass data between tasks?
A:
Use **XComs**:
python
CopyEdit
def push_data(**kwargs):
  kwargs['ti'].xcom_push(key='file_path', value='/data/file.csv')

def pull_data(**kwargs):
  path = kwargs['ti'].xcom_pull(key='file_path', task_ids='push_task')

✅ 206. Scenario: DAG Performance Tuning

Q: Your DAG takes too long to finish. How do you optimize it?
A:

- Enable task **parallelism** with proper depends_on_past=False.
- Set max_active_runs, concurrency, and pool limits.
- Run heavy tasks in **KubernetesPodOperator** or **Dataflow**.
- Avoid large XCom data.
- Break DAG into **subDAGs** or smaller DAGs for better scheduling.

✅ 207. Scenario: Handling Secrets in Cloud Composer

Q: How do you store secrets such as API keys?
A:

- Use **Secret Manager** and access via google.cloud.secretmanager.
- Use Airflow Connections for credentials (e.g., gcp_conn_id).
- Avoid hardcoding secrets in DAG files.

✅ 208. Scenario: Manually Trigger a DAG with Parameters

Q: How can you manually trigger a DAG and pass custom parameters?
A:
Via Airflow UI or API:
bash
CopyEdit
gcloud composer environments run my-env \
 --location us-central1 \

```
 dags trigger -- my_dag_id --conf '{"date":"2025-06-01"}'
```
In DAG:
python
CopyEdit
```
execution_date = kwargs['dag_run'].conf.get('date')
```

✅ 209. Scenario: Prevent DAG from Running in Parallel

Q: How would you ensure that a DAG only runs one instance at a time?
A:
Set max_active_runs=1 in the DAG definition:
python
CopyEdit
```
dag = DAG(
   dag_id="sequential_dag",
   max_active_runs=1,
   schedule_interval="@daily",
   ...
)
```

✅ 210. Scenario: Retry Policy Based on Exception Type

Q: You want to retry a task only when a specific exception occurs. How would you implement it?
A:
Use retry_on_failure condition:
python
CopyEdit
```
def custom_retry(e):
   return isinstance(e, TemporaryAPIError)

task = PythonOperator(
   task_id='task_with_custom_retry',
   python_callable=my_func,
   retries=3,
   retry_delay=timedelta(minutes=2),
   retry_exponential_backoff=True,
   retry_on_failure=custom_retry,
   dag=dag
)
```

✅ 211. Scenario: Run DAG for Custom Date Range

Q: Your DAG is scheduled daily, but you want to reprocess only specific dates. How can you achieve this?
A:
Use --conf parameters to pass dates and filter them inside the task:

bash
CopyEdit
airflow dags trigger my_dag --conf '{"start_date": "2025-05-01", "end_date": "2025-05-05"}'
In the Python task:
python
CopyEdit
def process(**kwargs):
  conf = kwargs['dag_run'].conf
  start = conf.get('start_date')
  end = conf.get('end_date')

✅ 212. Scenario: Skip Tasks Dynamically

Q: You want to skip certain tasks in a DAG based on a condition. How do you do that?
A:
Use ShortCircuitOperator or BranchPythonOperator:
python
CopyEdit
def should_run():
  if condition:
    return True
  return False

ShortCircuitOperator(
  task_id='check_condition',
  python_callable=should_run,
  dag=dag
)

✅ 213. Scenario: Organize Large DAG with Task Groups

Q: Your DAG has 50+ tasks. How do you make it easier to manage and view?
A:
Use **TaskGroup** for logical grouping:
python
CopyEdit
with TaskGroup("data_processing_group") as data_group:
  extract = BashOperator(...)
  transform = BashOperator(...)
  load = BashOperator(...)
  extract >> transform >> load

✅ 214. Scenario: Use KubernetesPodOperator in Composer

Q: You need to run a heavy Spark job inside Composer. How do you do this?

A:
Use KubernetesPodOperator:
python
CopyEdit
from airflow.providers.cncf.kubernetes.operators.kubernetes_pod import KubernetesPodOperator

```
task = KubernetesPodOperator(
    task_id="run_spark_job",
    namespace="default",
    image="gcr.io/my-project/spark-job:latest",
    cmds=["spark-submit", "--class", "Main", "job.jar"],
    dag=dag,
)
```

✅ 215. Scenario: Monitor Task and Send Slack Alert on Failure

Q: How would you send alerts to Slack when a task fails?
A:
Use on_failure_callback with Slack webhook:
python
CopyEdit

```
def slack_alert(context):
    import requests
    msg = f"Task Failed: {context['task_instance_key_str']}"
    requests.post("https://hooks.slack.com/services/XXX", json={"text": msg})

PythonOperator(
    task_id='alert_task',
    on_failure_callback=slack_alert,
    ...
)
```

✅ 216. Scenario: Avoid Hardcoding Configs

Q: How do you manage different environment configurations (dev, stage, prod) in DAGs?
A:
Use **Airflow Variables**, **Environment Variables**, or secrets:
python
CopyEdit

```
env = Variable.get("environment")  # dev / stage / prod
bucket = Variable.get(f"{env}_data_bucket")
```

✅ 217. Scenario: DAG is Skipping Runs Unexpectedly

Q: Your DAG is not running daily as expected. What could be wrong?

A:
Check for:
start_date in the future.
catchup=False when expecting past runs.
Misconfigured cron expression.
DAG paused in the UI.

✅ 218. Scenario: DAG File Not Detected by Airflow

Q: You created a new DAG file but it doesn't show up. Why?
A:
Possible reasons:
File name doesn't end with .py
Syntax errors in the DAG file
dag object not instantiated properly
File not present in the dags/ folder
File size >1MB (Composer has limits)

✅ 219. Scenario: Schedule DAG to Run Every Monday at 6 AM

Q: How do you schedule a DAG to run only on Mondays at 6 AM?
A:
Use a cron expression in the schedule_interval:
python
CopyEdit

```
dag = DAG(
    dag_id="weekly_monday_dag",
    schedule_interval="0 6 * * 1",  # 6 AM on Monday
    start_date=datetime(2025, 6, 2),
    catchup=False
)
```

✅ 220. Scenario: External Trigger with Parameters from API

Q: How do you trigger a DAG from an external system and pass runtime parameters?
A:
Call the Airflow REST API:
bash
CopyEdit

```
POST https://composer-env-url/dags/my_dag_id/dagRuns
{
  "conf": {
    "region": "us-west1",
    "run_type": "manual"
  }
}
```

Inside the DAG:

```
region = kwargs['dag_run'].conf.get('region')
```

✅ 221. Scenario: Task Timeout After a Certain Duration

Q: You want a task to fail if it runs more than 15 minutes. How do you enforce it?
A:
Use the execution_timeout parameter:

```
PythonOperator(
    task_id='long_running_task',
    python_callable=my_func,
    execution_timeout=timedelta(minutes=15),
    dag=dag
)
```

✅ 222. Scenario: Retry Exponential Backoff

Q: You want to retry a task, but with increasing intervals between attempts. How?
A:
Set:

```
retries=3,
retry_delay=timedelta(minutes=2),
retry_exponential_backoff=True
```

This will retry with delays like 2m, 4m, 8m.

✅ 223. Scenario: Transfer Files Between GCS Buckets

- **Q:** How do you copy files from one GCS bucket to another inside Airflow?
  A:
  Use GCSToGCSOperator:
  python
  CopyEdit

```
GCSToGCSOperator(
    task_id='copy_file',
    source_bucket='source-bucket',
    source_object='data/file.csv',
    destination_bucket='target-bucket',
    destination_object='archive/file.csv',
    move_object=False,
    dag=dag
)
```

✅ 224. Scenario: Use DAG-Level Default Arguments

- **Q:** How do you avoid repeating common arguments in all tasks?
  A:
  Use default_args dictionary:
  python
  CopyEdit

```
default_args = {
```

```
    'owner': 'data_team',
    'depends_on_past': False,
    'email_on_failure': True,
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
}

dag = DAG(
    dag_id='my_dag',
    default_args=default_args,
    schedule_interval='@daily',
    start_date=datetime(2025, 6, 1)
)
```

✅ 225. Scenario: Skip Backfilling of Old Dates

Q: You want a DAG to run only for current and future dates, not for the past. How?
A:
Set catchup=False in DAG:
python
CopyEdit

```
DAG(
    dag_id='no_backfill_dag',
    schedule_interval='@daily',
    catchup=False,
    start_date=datetime(2025, 6, 1)
)
```

✅ 226. Scenario: Store Logs Outside Composer

Q: You want Airflow logs to be stored in GCS. How do you configure it?
A:
Cloud Composer automatically stores logs in GCS under:
php-template
CopyEdit

```
gs://<composer-bucket>/logs/<dag_id>/<task_id>/<execution_date>/
```

You can configure log retention in Composer environment settings or bucket lifecycle policies.

✅ 227. Scenario: Load CSV from GCS to BigQuery

Q: How would you automate loading a CSV file from GCS into BigQuery in a DAG?
A:
Use GCSToBigQueryOperator:
python
CopyEdit

```
GCSToBigQueryOperator(
    task_id='load_csv_to_bq',
    bucket='my-bucket',
    source_objects=['data/my_file.csv'],
    destination_project_dataset_table='my_project.my_dataset.my_table',
    schema_fields=[{'name': 'id', 'type': 'INTEGER'}, {'name': 'name', 'type': 'STRING'}],
```

```
    skip_leading_rows=1,
    write_disposition='WRITE_TRUNCATE',
    source_format='CSV',
    dag=dag
)
```

✅ 228. Scenario: Upload File to GCS Using PythonOperator

Q: You want to upload a local file to GCS using a Python function in a DAG. How?
A:

```
python
CopyEdit
def upload_file_to_gcs():
    from google.cloud import storage
    client = storage.Client()
    bucket = client.get_bucket('my-bucket')
    blob = bucket.blob('data/output.csv')
    blob.upload_from_filename('/tmp/output.csv')

PythonOperator(
    task_id='upload_to_gcs',
    python_callable=upload_file_to_gcs,
    dag=dag
)
```

🔷 GCS (Google Cloud Storage) Real-Time Scenario Questions

✅ 229. Scenario: Transfer Files Between Projects

Q: You need to move files from Project A's bucket to Project B's bucket using a service account. How?
A:

- Grant Storage Object Admin on source to the service account.
- Use gsutil with impersonation:

```
bash
CopyEdit
gcloud auth activate-service-account --key-file=account.json
gsutil -u target-project cp gs://source-bucket/*.csv gs://target-bucket/
```

✅ 230. Scenario: Automate File Archival After 7 Days

Q: How do you auto-archive or delete files in GCS after 7 days?
A:
Use a **lifecycle rule**:

```
json
CopyEdit
{
 "rule": [
  {
   "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
   "condition": {"age": 7}
```

```
    }
  ]
}
```

Set via gsutil lifecycle set rule.json gs://my-bucket.

✅ 231. Scenario: Detect File Arrival in GCS for Pipeline Trigger

Q: You want to start a Composer DAG when a file lands in GCS. How?
A:

- Enable GCS → Pub/Sub notification.
- Create Cloud Function that listens to Pub/Sub and triggers Composer DAG using Airflow REST API.

✅ 232. Scenario: Upload CSV to GCS with Schema Validation

Q: How do you validate the schema of a CSV file before uploading it to GCS?
A:

- Use a local Python validation script (e.g., using pandas or csv module).
- If valid, upload with:
  python
  CopyEdit
  blob.upload_from_filename("validated_file.csv")

🔶 Dataform Real-Time Scenario Questions

✅ 23. Scenario: Model Fails Due to Table Not Existing

Q: A downstream model in Dataform fails because an upstream table was not created. How do you fix this?
A:

- Check dependencies or use ref() properly.
- Ensure upstream model is type: table or type: incremental.
- Add run_if: true only if it's conditional.

✅ 234. Scenario: Incremental Model with Change Data Capture

Q: How do you implement CDC (Change Data Capture) in a Dataform incremental model?
A:
sql
CopyEdit

```
config {
  type: "incremental",
  uniqueKey: "id",
  incrementalPreOperations: [
    "DELETE FROM ${self} WHERE id IN (SELECT id FROM ${ref('staging_table')})"
  ]
}

SELECT * FROM ${ref('staging_table')}
```

This ensures updated records are deleted and replaced in the incremental table.

✅ 235. Scenario: Dynamic Environment Config in Dataform

Q: You want to use different datasets for dev, stage, and prod. How?
A:
Use includes and environment blocks in dataform.json:
json
CopyEdit
"schemaSuffix": "${env}"
Or set in your config.js:
js
CopyEdit
module.exports = {
  defaultSchema: dataform.projectConfig.environment === "prod" ? "schema_prod" : "schema_dev"
}

✅ 236. Scenario: Schedule Dataform Workflow Daily

Q: How do you schedule a Dataform workflow to run every day?
A:
Use **Cloud Scheduler** or **Cloud Composer** to trigger Dataform via CLI/API:
bash
CopyEdit
dataform run --workflow daily_load --tag=prod

✅ 237. Scenario: Prevent Duplicate Data on Daily Load

Q: You load daily CSVs into a BigQuery table. Sometimes a day's file is reprocessed. How do you prevent duplicates?
A:

- Use MERGE statement with uniqueKey.
  sql
  CopyEdit
  MERGE INTO target_table T
  USING staging_table S
  ON T.id = S.id
  WHEN NOT MATCHED THEN
    INSERT (id, name) VALUES(S.id, S.name)
- Or use DISTINCT + INSERT:
  sql
  CopyEdit
  INSERT INTO target_table
  SELECT DISTINCT * FROM staging_table

✅ 238. Scenario: Query Only Latest Records

Q: How do you retrieve only the latest record per user based on timestamp?
A:
Use QUALIFY ROW_NUMBER():
sql
CopyEdit
SELECT * FROM user_activity
QUALIFY ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_time DESC) = 1

✅ 239. Scenario: Handle Semi-Structured JSON Data

Q: A GCS file contains JSON data with nested fields. How do you load and query it in BigQuery?
A:

- Load using source_format=NEWLINE_DELIMITED_JSON
- Query nested fields using dot notation:
  sql
  CopyEdit
  SELECT json_field.nested_key FROM dataset.table
- Use UNNEST() for arrays:
  sql
  CopyEdit
  SELECT id, value
  FROM dataset.table, UNNEST(array_field) AS value

🔶 GCS Real-Time Scenarios

✅ 240. Scenario: Enforce Data Retention and Compliance

Q: Your organization wants to auto-delete files older than 30 days. How?
A:
Apply a **lifecycle policy**:
json
CopyEdit
```
{
  "rule": [
    {
      "action": { "type": "Delete" },
      "condition": { "age": 30 }
    }
  ]
}
```
Apply via:
bash
CopyEdit
gsutil lifecycle set lifecycle.json gs://your-bucket

✅ 184. Scenario: Trigger Cloud Function on GCS Upload

Q: How do you trigger a Cloud Function when a file lands in GCS?
A:

- Enable **Object Finalize** event.
- Create function:
  python
  CopyEdit
  def gcs_trigger(event, context):
  bucket = event['bucket']
  name = event['name']
  print(f"New file: {name} in {bucket}")
- Deploy with:
  bash
  CopyEdit
  gcloud functions deploy gcs_trigger \
  --runtime python39 \
  --trigger-resource your-bucket \
  --trigger-event google.storage.object.finalize

🔷 **Dataform Real-Time Scenarios**

✅ **241. Scenario: Model Reuse Across Workspaces**
Q: You want to reuse models or logic across multiple Dataform repositories. How?
A:

- Use **includes** to import SQLX files from a shared repo.
- Define common CTEs or functions in includes/common.sqlx and reference via:
  sql
  CopyEdit
  config { type: "view" }

  include "includes/common.sqlx";

  SELECT * FROM ${ref("base_table")}

✅ **242. Scenario: Test a Transformation Logic**
Q: You want to test whether a transformation correctly filters out invalid data. How?
A:
Create a **Dataform test**:
sql
CopyEdit
config {
type: "test",
tags: ["qa"]
}

SELECT * FROM ${ref("cleaned_data")}
WHERE is_valid = FALSE
If rows are returned, the test fails.

✅ **242. Scenario: Create a Parametrized Table Name**
Q: You want your table to dynamically adapt to an environment (e.g., dev/prod). How?
A:
Use schemaSuffix or config injection:
js
CopyEdit
config {
  schema: `dataset_${dataform.projectConfig.environment}`
}

✅ **243. Scenario: Run DAG for Specific Subset of Models**
Q: You only want to run a subset of models with the tag daily. How?
A:
Use CLI:
bash
CopyEdit
dataform run --tags=daily
Or in workflow config:
json
CopyEdit
{
  "includedTags": ["daily"]
}
✅ **244. Scenario: Trigger a DAG from Another DAG**
Q: You have two dependent DAGs. DAG B should run only after DAG A finishes. How do you trigger DAG B from DAG A?
A:
Use TriggerDagRunOperator in DAG A:
python
CopyEdit
TriggerDagRunOperator(
    task_id='trigger_dag_b',
    trigger_dag_id='dag_b',
    dag=dag_a
)
Or use **ExternalTaskSensor** in DAG B to wait for DAG A:
python
CopyEdit
ExternalTaskSensor(
    task_id='wait_for_dag_a',
    external_dag_id='dag_a',
    external_task_id='final_task_of_dag_a',
    dag=dag_b
)

✅ **245. Scenario: Monitor a File in GCS and Trigger DAG**

Q: You want to start a DAG only when a specific file arrives in GCS. How do you implement this?
A:
Use GCSObjectExistenceSensor:
python
CopyEdit
```
from airflow.providers.google.cloud.sensors.gcs import GCSObjectExistenceSensor

wait_for_file = GCSObjectExistenceSensor(
    task_id='wait_for_file',
    bucket='my-bucket',
    object='data/input_file.csv',
    timeout=600,
    poke_interval=30,
    dag=dag
)
```
✅ **246. Scenario: Passing Data Between Tasks**
Q: You need to pass output from Task A to Task B in the same DAG. How can you achieve this?
A:
Use **XCom** to pass data between tasks:
python
CopyEdit
```
def task_a(**kwargs):
    kwargs['ti'].xcom_push(key='result', value='processed_value')

def task_b(**kwargs):
    result = kwargs['ti'].xcom_pull(key='result', task_ids='task_a')
    print(result)
```
Use provide_context=True or op_kwargs={"key": "value"} for context passing
2nd Par

1. **How does Bigquery Works?**
    - Bigquery Storage
    - Bigquery Analytics
    - Bigquery Administration
2. **What is Dataset in Bigquery?**

    Datasets are top-level containers that are used to organize and control access to your tables and views. A table or view mush belong to a dataset.

3. **What types of tables are supported by Bigquery?**
    Native tables
    External tables
    Views
4. **what is Partition Pruning?**
    If a query uses a qualifying filter on the value of the partitioning column, Bigquery can scan the partitions that match the filter and skip the remaining partitions. This process is called partition pruning.

5. **what are the best practices for partition pruning ?**
   Use - Use a constant filter expression
   Isolate - Isolate the partition column in your filter.
   Require – Require a partition filter in queries.
6. **What is google Bog Query?**
7. **How does google big query handle scalling?**
   BigQuery automatically scales to handles any size of data without the need for manual intervention it achieves this by dynamically.
   Allocating compute resources based on the size and complexity of the query being executed.
8. **What are the key features of Google Bigquery ?**
   - Bigquery offers features such as
   - Server less infrastructure.
   - Support standard SQL queries
   - Real-time analytics
   - Integration with other GCP services.
   - Automatic Scaling and High Availability
9. **What Is the maximum size of a dataset in BigQuery?**
   Bigquery supports datasets of up to 20TB in size for querying using standard SQL However, users can store and manage larger datasets by partitioning tables or using federated queries to access data stored externally.
10. **How does Bigquery handle Data Security?**
    Bigquery provides several security features, Including data encryption at rest and in transit, Identity and access management (IAM) controls, audit logging and integration with Google Cloud Security Command centre.
11. **Explain how the partitions work in bigquery?**
    Partitioning in BigQuery involves dividing the large table into smaller , Manageable Partitions based on specific column(eg – date) this improves query performance and reduces costs by limiting the amount of data scanned during queries.

12. **What is the difference between a table and view in Bigquery?**
    A table in Bigquery stores structured data in tabular format. Where a view virtual table defined by a SQL query views allows users to encapsulate complex logic and access subsets of data without duplicating storage .
13. **How can you optimize query performance in BigQuery?**
    Query performance in Bigquery can be optimized by using partitioned tables, clustering, denormalization, caching frequently accessed data, avoiding SELECT * from .
14. **How does Bigquery handle nested and repeated fields in JSON data.**
    BigQuery supports nested and repeated fields in JSON data, Allowing users to query and analyze hierarchical data structures and nested fields are accessed using **dot notation,** while Repeated fields are treated as **Arrays.**
15. **Explain the concepts of slots in Bigquery?**
    Slots in Bigquery represent the computational resources allocated to executed queries
16. **How can you export data from BigQuery to external storage or services ?**
    Data can be exported from Bigquery to external storage or services using various methods, Including Bigquery export jobs, Google cloud storage (GCS) exports, federated queries and BigQuery Data Transfer Services.
17. **What is the maximum duration for running a single query in BigQuery?**

The maximum duration for running a single query in BigQuery is 6 hr Queries exceeding this duration automatically aborted to prevent resource exhaustion.

18. **Explain the difference between streaming insert and batch inserts in Bigquery?**
    Streaming :- Streaming inserts allows real time data ingestion into BigQuery table using BigQuery Streaming API
    Batch Load :- Where as batch inserts involve loading data into bigquery tables from external storage using batch jobs or data transfer services.
19. **How does BigQuery Handle data deduplication?**
    BigQuery automatically deduplicates data during query execution when using standard SQL queries Duplicate rows are removed based on their quality, and only unique rows are returned in query results .
20. **What are the limitation of BigQuery?**
    Some limitations of BigQuery include maximum table and partition sizes, query execution time limits, storage retention policies
21. **Explain the concepts of slots reservation in Bigquery?**
    Slot reservation in Bigquery allows users to reserve a fix number of slots for their project, ensuring guaranteed query throughput and concurrency reserved slots are billed at a discounted rate compared to on- demand slots.
22. **What is difference between cache hit and cache miss in Bigquery ?**
    A cache hit occurs when a query result is retrieved from the query cache, avoiding the need for query execution.
    A cache miss occurs when a query result is not found in the cache and much be recomputed by executing the query.
23. **How does Bigquery handle data ingestion from external sources?**
    BigQuery support data ingestion from various external sources, GCS , Google Cloud Datastore , Pub/Sub DataPrep.
24. **Explain the concept of table clustering in BigQuery?**
    Table clustering in BigQuery involves organizing data within a table based on one or more columns.it improves query performance by physically reordering the data in storage based on the cluster keys, which are specified during table creation or modification.

Clustering helps reduce the amount of data scanned during queries by grouping related rows together, leading to faster query execution and lower costs.

25. **How does BigQuery handle query optimization?**
    Bigquery optimizes queries automatically by analysing query patterns
26. **What is the difference between petitioned tables and clustered tables in BigQuery?**
    Partitioned tables in BigQuery are divided into segments based on a specified column value, Which improve query performance and reduces costs.
    Clustered tables, on the other hand, physically organize data with in partitions based on one or more clustering columns.
27. **How does BigQuery handle schema updates for tables with streaming data?**
    BigQuery supports schema updates for tables with streaming data without interrupting data ingestion new fields can be added to the schema automatically, and existing fields can be modified or deleted while maintaining data integrity and availability.
28. **How you can schedule recurring queries in BigQuery?**
    Recurring queries in BigQuery can be scheduled using the scheduled query future, which allows user to specify a SQL query and schedule its execution at regular intervals.
29. **What is the purpose of data retention policies in BigQuery?**

Data retention policy in BigQuery specify how long data should be retained in storage before being deleted

30. **Explain the concept of Materialized View in BigQuery?**
    Materialized Views in BigQuery are precomputed query results stored as tables. They may improve the query performance by caching intermediate results and reducing the need to recompute expensive queries.

    Materialized views are automatically refreshed to reflect changes in underlying data.
31. **Explain how cache works in BigQuery?**
    BigQuery caches query results to improve query performance and reduce costs. Query results are cached for a certain period, and subsequent identical queries.
32.
33.
34.
35.